



**OSTIM TECHNICAL UNIVERSITY
FACULTÉ D'INGÉNIERIE
RAPPORT DE PROJET DE FIN D'ÉTUDES**

**ASTROBOT — UN ASSISTANT CONVERSATIONNEL
INTELLIGENT À CAPACITÉS MULTIMODALES, GÉNÉRATION
AUGMENTÉE PAR LA RECHERCHE (RAG) ET INTÉGRATION DE
WORKFLOWS WEB**

Équipe du projet

TIDIANE KONATE (220201838)
SIDI MOHAMED SALL (210208992)
AHMED ESSALEM (220201992)
SAAD IBRAHIM HOUSSEIN (210201983)

Encadrant du projet

Assist. Prof. Dr. Yücel TEKİN

Projet de licence

Département de Génie Informatique
MFBP 402 — Projet de Fin d'Études II

Mai 2026

APPROBATION

Ce projet de fin d'études de licence, intitulé « AstroBot — Un assistant conversationnel intelligent à capacités multimodales, génération augmentée par la recherche (RAG) et intégration de workflows web », préparé par Tidiane Konate, Sidi Mohamed Sall, Ahmed Essalem et Saad Ibrahim Houssein sous la supervision de Assist. Prof. Dr. Yücel TEKİN, a été examiné par le jury et accepté comme projet de fin d'études de licence en termes de portée et de qualité.

Assist. Prof. Dr. Yücel TEKİN

Encadrant

Nom Prénom du membre du jury

Membre du jury

Nom Prénom du membre du jury

Membre du jury

Nom Prénom du chef de département

Chef de département

REMERCIEMENTS

Les auteurs remercient chaleureusement leur encadrant de projet, Assist. Prof. Dr. Yücel TEKİN, dont les retours lors des revues ont façonné aussi bien la portée que la qualité de ce travail. Nous remercions le Département de Génie Informatique de l'OSTİM Technical University pour les ressources mises à disposition tout au long du projet, ainsi que nos camarades qui ont participé à des séances informelles de tests d'utilisabilité et nous ont aidés à affiner l'interface et le comportement de l'assistant. Enfin, nous remercions les mainteneurs des outils open source et des services à offre gratuite sur lesquels AstroBot est construit.

ÖZET (RÉSUMÉ EN TURC)

Bu çalışma, AstroBot adlı çok modlu yeteneklere sahip akıllı bir konuşma tabanlı yapay zekâ asistanının tasarımı ve gerçekleştirimini sunmaktadır. Proje, günümüz yapay zekâ araçlarının dağınıklığı sorununu ele alır: sohbet, görsel üretimi, belge arama, OCR ve ses dökümü genellikle ayrı uygulamalarda yer alır ve kullanıcılar en güncel modellere erişmek için birden çok ücretli aboneliğe ihtiyaç duyar. Önerilen sistem; herkese açık bir sohbet arayüzü, bir yönetici paneli ve her kullanıcı mesajını Mistral büyük dil modeline yönlendiren n8n tabanlı bir iş akışı düzenleyicisinden oluşur. Sistem; metin tabanlı sohbet, görsel üretimi, görselden görsele düzenleme, OCR ve görüntü altyazılama, ses dökümü, belgeler üzerinde soru-cevap (RAG), PDF rapor üretimi ve tekrar kullanılabilir iş akışı zincirleri gibi temel işlevleri içerir. Geliştirme süreci yinelemeli sprint'ler hâlinde yürütülmüş, gereksinimler son kullanıcı ve yönetici paydaşlarından elde edilmiştir. Çözüm Docker ile konteynerleştirilmiş, gerçek bir sunucu üzerinde dağıtılmış ve 66 entegrasyon testinden oluşan bir paketle doğrulanmıştır.

Anahtar Kelimeler: Konuşma tabanlı yapay zekâ, büyük dil modeli, RAG, çok modluluk, iş akışı düzenleme, MVP

RÉSUMÉ

Ce rapport présente la conception et l'implémentation d'AstroBot, un assistant conversationnel intelligent fondé sur l'intelligence artificielle et doté de capacités multimodales. Le projet répond à la fragmentation des outils d'IA actuels : le chat, la génération d'images, la recherche dans des documents, l'OCR et la transcription audio résident généralement dans des applications distinctes, et les utilisateurs doivent jongler avec plusieurs abonnements payants pour accéder aux modèles de pointe. Le système proposé se compose d'une interface publique de chat, d'un tableau de bord d'administration et d'un orchestrateur de workflows basé sur n8n qui achemine chaque message utilisateur via un grand modèle de langage Mistral. Le produit minimum viable (MVP) couvre les fonctionnalités principales : chat textuel avec mémoire conversationnelle et personas, génération d'images et retouche image-à-image, OCR et légendage visuel, transcription audio, questions-réponses augmentées par la recherche (RAG) sur des documents PDF téléversés, génération de rapports PDF à la demande, ainsi que des chaînes de workflows réutilisables, le tout aux côtés d'un service d'administration distinct pour les statistiques et la gestion des utilisateurs. Le développement a suivi un processus itératif de type Scrum sur quatre sprints, les besoins ayant été recueillis auprès des utilisateurs finaux et des administrateurs. La solution est conteneurisée avec Docker, déployée sur un serveur virtuel en production, et validée par une suite de 66 cas de tests d'intégration. L'évaluation montre que le système satisfait ses critères d'acceptation et atteint l'objectif de réunir plusieurs modalités d'IA derrière une seule application web gratuite.

Mots-clés : IA conversationnelle, grand modèle de langage, génération augmentée par la recherche, multimodalité, orchestration de workflows, MVP

TABLE DES MATIÈRES

	Page
TABLE DES MATIÈRES	iv
LISTE DES TABLEAUX.....	vi
LISTE DES FIGURES.....	vii
LISTE DES SYMBOLES ET ABRÉVIATIONS.....	viii
1. Introduction.....	1
1.1. Énoncé du problème.....	1
1.2. Objectifs du projet.....	1
1.3. Portée du projet	2
1.4. Organisation du rapport	2
2. Concepts généraux	3
2.1. Définition du domaine	3
2.2. Technologies utilisées.....	3
2.3. Concepts fondamentaux.....	4
3. Travaux connexes.....	5
3.1. Systèmes existants.....	5
3.2. Études académiques	5
3.3. Analyse comparative.....	5
4. Modèle de processus logiciel.....	7
4.1. Modèle de développement choisi.....	7
4.2. Définition des rôles Scrum.....	7
4.3. Processus Scrum	8
4.4. Mise en œuvre de Scrum dans le projet	8
5. Analyse des besoins	10
5.1. Parties prenantes	10
5.2. Besoins utilisateurs	10
5.2.1. Besoins fonctionnels.....	10
5.2.2. Besoins non fonctionnels.....	11
5.3. Spécifications du système	11
6. Modélisation du système.....	13
6.1. Diagramme de contexte	13
6.2. Diagramme de cas d'utilisation	13
6.3. Descriptions des cas d'utilisation	14
6.4. Diagrammes de séquence.....	15
7. Architecture et conception du système.....	17

7.1. Architecture du système.....	17
7.2. Modèles structurels	18
7.2.1. Diagramme de classes / entité-association.....	19
7.2.2. Héritage (généralisation).....	19
7.2.3. Agrégation / composition.....	20
7.3. Modèles comportementaux	20
7.3.1. Modèle événementiel	20
7.3.2. Diagramme d'états.....	20
8. Implémentation du système.....	22
8.1. Environnement de développement.....	22
8.2. Composants du système	22
8.3. Structure du code	23
9. Développement du MVP	25
9.1. Définition du MVP.....	25
9.2. Fonctionnalités implémentées.....	25
9.3. Interfaces du système	26
9.4. Démonstration du système	30
10. Tests et évaluation	33
10.1. Stratégie de tests.....	33
10.2. Scénarios de tests	33
10.3. Résultats des tests.....	33
11. Résultats et discussion.....	35
12. Conclusion et travaux futurs	36
12.1. Conclusion	36
12.2. Travaux futurs	36
RÉFÉRENCES	37
CURRICULUM VITAE	38
13. Annexes	46
13.1. Annexe A : Code source	46
13.2. Annexe B : Backlog des sprints	48
13.3. Annexe C : Guide utilisateur.....	49

LISTE DES TABLEAUX

	Page
Tableau 4.1 : Rôles Scrum et membres de l'équipe les ayant assumés	7
Tableau 4.2 : Plan des sprints et incréments livrés.....	8
Tableau 5.1 : Besoins fonctionnels	10
Tableau 5.2 : Besoins non fonctionnels	11
Tableau 5.3 : Endpoints REST sélectionnés et leurs responsabilités	11
Tableau 7.1 : Principales tables de la base de données et leur rôle	18
Tableau 8.1 : Services tiers utilisés par AstroBot	22
Tableau 10.1 : Suite de tests d'intégration — résultats par catégorie.....	33

LISTE DES FIGURES

	Page
Figure 6.1 : Diagramme de contexte de la plateforme AstroBot.....	13
Figure 6.2 : Diagramme de cas d'utilisation (utilisateur final et administrateur)	14
Figure 6.3 : Diagramme de séquence d'un aller-retour de message de chat.....	16
Figure 7.1 : Architecture du système (couches client / application / orchestration / données).....	18
Figure 7.2 : Modèle entité-association du schéma PostgreSQL.....	19
Figure 7.3 : Diagramme d'états d'un message de conversation	21
Figure 8.1 : Workflow n8n qui achemine chaque message via Mistral	23
Figure 9.1 : Page d'accueil publique avec la barre de chat centrale	26
Figure 9.2 : Section fonctionnalités de l'application publique	27
Figure 9.3 : Page « À propos » présentant l'équipe de quatre développeurs	27
Figure 9.4 : Interface de chat authentifiée (écran d'accueil).....	28
Figure 9.5 : Sélecteur de personas — sept personas d'assistant disponibles	28
Figure 9.6 : Paramètres de profil avec un recadreur d'image dans le navigateur	28
Figure 9.7 : Tableau de bord admin — KPI et graphiques d'activité	29
Figure 9.8 : Tableau de gestion des utilisateurs (admin).....	29
Figure 9.9 : Paramètres en libre-service de l'admin (profil et mot de passe)	30
Figure 9.10 : Réponse riche en Markdown rendue dans une bulle de l'assistant	30
Figure 9.11 : Exemple de génération d'image par IA.....	31
Figure 9.12 : Rapport PDF à la demande affiché comme carte de téléchargement	31
Figure 9.13 : Analyse OCR d'une image attachée.....	31
Figure 9.14 : Questions-réponses sur document — téléversement et liaison d'un PDF (notebook / RAG).....	32
Figure 9.15 : Composition d'une chaîne de workflow réutilisable.....	32
Figure 13.1 : Chronologie des sprints du projet	49

LISTE DES SYMBOLES ET ABRÉVIATIONS

IA : Intelligence Artificielle

API : Application Programming Interface (interface de programmation)

BLIP : Bootstrapping Language-Image Pre-training (modèle de légendage d'images)

CRUD : Create, Read, Update, Delete (créer, lire, mettre à jour, supprimer)

CSS : Cascading Style Sheets (feuilles de style en cascade)

BD : Base de Données

ETA : Estimated Time of Arrival (utilisé de manière générique)

BF : Besoin Fonctionnel

HTML : HyperText Markup Language

HTTP : HyperText Transfer Protocol

JSON : JavaScript Object Notation

JWT : JSON Web Token (jeton web JSON)

KaTeX : Bibliothèque web de composition mathématique LaTeX

LLM : Large Language Model (grand modèle de langage)

MVP : Minimum Viable Product (produit minimum viable)

BNF : Besoin Non Fonctionnel

OCR : Optical Character Recognition (reconnaissance optique de caractères)

ORM : Object-Relational Mapping (mapping objet-relationnel)

RAG : Retrieval-Augmented Generation (génération augmentée par la recherche)

REST : Representational State Transfer

SQL : Structured Query Language

STT : Speech-to-Text (parole vers texte)

TTS : Text-to-Speech (texte vers parole)

UI / UX : User Interface / User Experience (interface / expérience utilisateur)

UML : Unified Modeling Language

VM : Virtual Machine (machine virtuelle)

VPS : Virtual Private Server (serveur privé virtuel)

1. Introduction

1.1. Énoncé du problème

Les assistants d'intelligence artificielle modernes sont puissants mais fragmentés. Un utilisateur qui souhaite discuter avec un modèle, générer une image, transcrire un enregistrement audio, lancer une reconnaissance optique de caractères sur une capture d'écran et poser des questions sur un fichier PDF doit généralement utiliser quatre ou cinq applications différentes, chacune avec son propre compte, sa propre interface et — de plus en plus — son propre abonnement payant. Les modèles de pointe se trouvent derrière des barrières payantes qui désavantagent les étudiants et les petites équipes, tandis que l'auto-hébergement de modèles open source reste complexe car chaque modèle a sa propre histoire d'intégration. Les conversations sont également traitées comme jetables : la plupart des outils n'offrent que de simples listes de messages, sans dossiers, sans étiquettes, sans moyen d'épingler un document comme contexte durable, et sans trace exportable de ce qui a été dit. Les pipelines réutilisables de prompts à plusieurs étapes sont soit reconstruits à chaque fois, soit cachés dans des agents fermés. Il n'existe pas de lieu unique et abordable qui réunit l'IA conversationnelle, l'entrée multimodale, la compréhension de documents et les workflows réutilisables.

1.2. Objectifs du projet

L'objectif de ce projet est de concevoir, implémenter et déployer un produit minimum viable (MVP) qui unifie ces capacités derrière une seule application web gratuite. Les objectifs spécifiques sont :

- Offrir une surface de chat conversationnel adossée à un grand modèle de langage, avec une mémoire par session, plusieurs personas et un rendu enrichi du Markdown, des mathématiques, du code et des diagrammes.
- Prendre en charge l'entrée et la sortie multimodales — génération d'images et retouche image-à-image, OCR et légendage visuel, et transcription audio.
- Offrir la compréhension de documents via la génération augmentée par la recherche (RAG) sur des PDF téléversés, ainsi qu'un mode notebook qui épingle un document comme contexte durable.
- Rendre les conversations persistantes et exportables, avec des dossiers/étiquettes, une recherche et la génération de rapports PDF.

- Fournir un tableau de bord d'administration pour les statistiques, la gestion des utilisateurs et l'audit des conversations, isolé de l'application publique.
- Maintenir le système gratuit à exploiter en combinant des APIs à offre gratuite (Mistral, HuggingFace, Groq) et un moteur de workflows auto-hébergeable, et le livrer comme un déploiement Docker reproductible.

1.3. Portée du projet

Le projet couvre le pipeline complet de génie logiciel pour le MVP : analyse des besoins, modélisation du système, architecture et conception, implémentation, tests et déploiement sur un serveur en production. Sont inclus l'application publique de chat, le tableau de bord d'administration, le workflow d'orchestration n8n et les douze-et-plus fonctionnalités basées sur l'IA décrites dans ce rapport. Sont hors portée — et reportés aux travaux futurs — les applications mobiles natives, un modèle interne ajusté finement, la facturation et l'application de quotas, et les tests de charge à grande échelle. Le système vise un environnement de déploiement unique plutôt qu'une offre SaaS multi-tenants.

1.4. Organisation du rapport

La section 2 introduit le domaine et les technologies habilitantes. La section 3 passe en revue les systèmes existants et les travaux académiques et positionne AstroBot parmi eux. La section 4 décrit le modèle de processus logiciel. La section 5 présente l'analyse des besoins, et la section 6 les modèles du système (diagrammes de contexte, de cas d'utilisation et de séquence). La section 7 détaille l'architecture ainsi que la conception structurelle et comportementale. La section 8 couvre l'implémentation ; la section 9 le MVP, ses fonctionnalités, ses interfaces et une démonstration. La section 10 présente les tests et l'évaluation, la section 11 discute les résultats, et la section 12 conclut avec les travaux futurs. Les curriculum vitae des membres de l'équipe et les annexes (extraits de code source, backlog des sprints et guide utilisateur) suivent les références.

2. Concepts généraux

2.1. Définition du domaine

AstroBot appartient au domaine des applications d'IA conversationnelle : des systèmes logiciels qui exposent un grand modèle de langage via une interface de chat et orchestrent des outils complémentaires — modèles d'images, modèles de parole, recherche et récupération — pour répondre aux demandes des utilisateurs. Ce domaine combine la conception d'interactions en langage naturel, le génie des applications web et les préoccupations opérationnelles liées à l'appel fiable et économique d'APIs de modèles externes. À l'intérieur de ce domaine, le projet se situe à l'intersection d'un outil de productivité pour utilisateur final et d'une petite plateforme opérationnelle disposant de son propre volet d'administration.

2.2. Technologies utilisées

La pile technique est délibérément construite à partir de briques bien soutenues et majoritairement gratuites :

- Front-end — HTML, CSS et JavaScript natifs (sans framework), avec KaTeX pour les mathématiques, highlight.js pour le code, Mermaid pour les diagrammes, Cropper.js pour le recadrage d'avatar, et Chart.js dans le tableau de bord d'administration.
- Back-end — Node.js 18 avec Express 4, JSON Web Tokens pour l'authentification, bcrypt pour le hachage des mots de passe, Helmet pour les en-têtes de sécurité HTTP, et express-rate-limit. Le traitement des documents et des PDF utilise pdf-parse et PDFKit ; l'OCR utilise Tesseract.js.
- IA et modèles — Mistral medium-latest comme modèle de chat (atteint via l'AI Agent de n8n), HuggingFace Inference pour FLUX.1-schnell (génération d'images), BLIP (légendage d'images) et MiniLM-L6-v2 (embeddings de phrases), Groq Whisper-large-v3 pour la transcription, et SerpAPI pour la recherche web en direct au sein de l'agent.
- Orchestration et infrastructure — n8n comme moteur visuel de workflows, PostgreSQL (avec l'image pgvector, utilisée comme simple base relationnelle) pour la persistance, et Docker avec Docker Compose pour l'empaquetage et le déploiement. Un script Python (deploy.py) effectue le déploiement par SSH/SFTP sur le serveur.

2.3. Concepts fondamentaux

Plusieurs concepts reviennent tout au long du rapport. Un grand modèle de langage (LLM) est un réseau de neurones entraîné à prédire du texte ; étant donné une conversation, il en produit la suite, et avec un prompt système et des outils, il peut agir comme un agent. La génération augmentée par la recherche (RAG) améliore l'ancrage en récupérant les fragments les plus pertinents d'un document — ici classés par similarité cosinus sur des embeddings MiniLM — et en les insérant dans le prompt avant que le modèle ne réponde. L'orchestration de workflows consiste à décrire un pipeline à plusieurs étapes (recevoir une requête, la router selon son type, l'enrichir, appeler le modèle, post-traiter, répondre) comme un graphe modifiable sans redéployer le code applicatif ; AstroBot utilise n8n pour cela. Les JSON Web Tokens portent une identité signée et à durée de vie limitée, de sorte que l'API sans état peut autoriser les requêtes, et une revendication `is_admin` verrouille le service d'administration. La conteneurisation empaquette chaque service avec ses dépendances afin que le système soit reproductible à partir d'une machine virtuelle vierge.

3. Travaux connexes

3.1. Systèmes existants

Les assistants généralistes tels que ChatGPT, Claude et Gemini offrent une excellente qualité conversationnelle et, dans leurs offres payantes, certaines fonctionnalités multimodales. Cependant, les capacités avancées — grandes fenêtres de contexte, génération d'images, analyse de documents — sont généralement réservées aux abonnements, et les tâches plus spécialisées (OCR, transcription, recherche vectorielle sur des documents privés, pipelines reproductibles à plusieurs étapes) sont réparties entre des produits distincts tels que des services OCR dédiés, des APIs de parole-vers-texte et des outils RAG de type notebook. Des alternatives auto-hébergeables (modèles ouverts servis localement) existent mais demandent un effort opérationnel important et offrent rarement une interface multifonctionnelle aboutie clé en main.

3.2. Études académiques

Les travaux sur la génération augmentée par la recherche ont montré que ancrer les sorties du modèle dans des passages récupérés réduit les hallucinations et améliore l'exactitude factuelle pour la question-réponse en domaine ouvert, et que les récupérateurs denses fondés sur des embeddings de type transformeur avec similarité cosinus surpassent généralement les références lexicales creuses dans ce cadre. Les travaux sur les agents fondés sur des LLM démontrent que donner à un modèle un petit ensemble d'outils (recherche, brouillon de raisonnement, contrat de sortie structurée) lui permet de décomposer les tâches de manière fiable tout en gardant le système environnant simple. Du côté du génie logiciel, des études sur la livraison agile rapportent que le développement itératif piloté par les retours utilisateurs convient bien aux projets dont les besoins évoluent à mesure que les utilisateurs essaient les premières versions — ce qui est précisément la situation d'un assistant riche en fonctionnalités.

3.3. Analyse comparative

Comparé aux alternatives ci-dessus, AstroBot met en avant trois éléments distinctifs. Premièrement, l'étendue derrière une seule surface : chat, génération et retouche d'images, OCR et légendage, transcription, RAG sur documents, mode notebook, génération de rapports PDF, chaînes de workflows et volet d'administration sont tous accessibles depuis une même application avec un seul compte. Deuxièmement, l'accessibilité financière : en combinant des APIs de modèles à offre gratuite avec une couche d'orchestration auto-

hébergeable, le système est gratuit à exploiter pour l'équipe et ses utilisateurs. Troisièmement, la transparence et la reproductibilité : la logique d'orchestration est un graphe n8n visible qui peut être ajusté sans redéploier le code, le modèle de données et le déploiement sont documentés, et toute la pile démarre avec une seule commande docker compose. AstroBot ne cherche pas à battre les modèles de pointe sur la qualité brute ; il intègre bien des modèles accessibles.

4. Modèle de processus logiciel

4.1. Modèle de développement choisi

Un modèle agile itératif de type Scrum a été retenu. L'ensemble des fonctionnalités souhaitables pour un assistant IA est vaste et évolue rapidement à mesure que l'équipe et les testeurs informels utilisent les premières versions, si bien qu'un processus en cascade piloté par un plan aurait été mal adapté. Le projet a été organisé en quatre sprints courts, chacun produisant un incrément fonctionnel et testable du produit, avec les éléments les plus précieux et les plus fondateurs planifiés en premier.

4.2. Définition des rôles Scrum

Le projet étant mené par une équipe étudiante de quatre personnes, les rôles Scrum standards ont été projetés sur l'équipe comme le montre le Tableau 4.1. Le rôle de Product Owner (priorisation du backlog, validation des incréments selon les critères d'acceptation) a été tenu principalement par le chef de projet en concertation avec l'encadrant, qui a joué le rôle de partie prenante externe. Le rôle de Scrum Master (animation des cérémonies, levée des blocages) a tourné de manière informelle mais est resté ancré sur le chef de projet. Les quatre membres formaient l'équipe de développement, chacun étant responsable d'un ou plusieurs domaines fonctionnels.

Tableau 4.1 : Rôles Scrum et membres de l'équipe les ayant assumés.

Rôle Scrum	Tenu par	Principales responsabilités
Product Owner / partie prenante	Tidiane Konaté (avec l'encadrant)	Priorité du backlog, acceptation des incréments, décisions de portée
Scrum Master	Tidiane Konaté	Cérémonies de sprint, levée des blocages, coordination
Équipe de développement — Architecture / Back-end / Déploiement	Tidiane Konaté	Architecture système, API REST, workflow n8n, Docker, déploiement
Équipe de développement — Vision / Produit / UX	Sidi Mohamed Sall	Cadrage produit, conception de l'interface publique, séances d'utilisabilité
Équipe de développement — IA / LLM / Multimodal	Ahmed Essalem	Intégration de modèles, fonctionnalités image/OCR/audio, pipeline RAG
Équipe de développement — Tests / Front-end / QA	Saad Ibrahim Houssein	Finition du front-end, tableau de bord admin, suite de tests d'intégration

4.3. Processus Scrum

Chaque sprint a suivi les cérémonies habituelles, adaptées à la taille de l'équipe : une réunion de planification de sprint pour sélectionner les éléments du backlog et définir les critères d'acceptation, de courtes synchronisations quotidiennes pour faire émerger l'avancement et les blocages, une revue de sprint au cours de laquelle l'incrément était démontré, et une brève rétrospective pour ajuster la manière de travailler. Le backlog produit était maintenu comme une liste de récits utilisateurs avec une priorité et des critères d'acceptation ; les éléments non terminés dans un sprint étaient repriorisés pour le suivant. Les retours de revue de l'encadrant alimentaient directement le backlog.

4.4. Mise en œuvre de Scrum dans le projet

Le projet a été livré en quatre sprints sur environ douze semaines, comme résumé dans le Tableau 4.2 et la Figure 13.1 (Annexe B). Le sprint 1 a posé les fondations — dépôt, Docker Compose, schéma de base de données, authentification JWT et un chemin de chat minimal via n8n. Le sprint 2 a construit le cœur conversationnel — l'AI Agent Mistral, l'historique de conversation, les personas, le rendu enrichi du contenu et la révélation de style streaming dans l'interface de chat. Le sprint 3 a ajouté les fonctionnalités multimodales et de récupération — génération d'images et retouche image-à-image, OCR et légendage BLIP, transcription Whisper, RAG et mode notebook, et rapports PDF à la demande. Le sprint 4 a livré le tableau de bord d'administration, la suite de 66 tests d'intégration, le script de déploiement, le durcissement de sécurité et la documentation.

Tableau 4.2 : Plan des sprints et incréments livrés.

Sprint	Semaines	Objectif	Incrément livré
1 — Fondations	≈ 1–3	Mettre en place le squelette	Dépôt, Docker Compose, schéma PostgreSQL, authentification JWT, chat de base via n8n
2 — Cœur conversationnel	≈ 3–5	Rendre le chat de qualité	AI Agent Mistral, historique de conversation, 7 personas, Markdown/KaTeX/highlight.js/Mermaid, révélation de style streaming
3 — Multimodal et RAG	≈ 5–8	Ajouter les fonctionnalités IA	Génération d'images et img-à-img, OCR, légendes BLIP, transcription Whisper, RAG et mode notebook, génération de rapports PDF, chaînes de workflows, étiquettes/exports
4 — Admin, tests et déploiement	≈ 8–11	Exploiter et durcir	Tableau de bord admin (stats, utilisateurs, audit), suite de 66 tests d'intégration, deploy.py, durcissement de sécurité, documentation

5. Analyse des besoins

5.1. Parties prenantes

Les parties prenantes principales sont les utilisateurs finaux de l'application publique — étudiants préparant des rapports et du code, développeurs explorant l'IA multimodale, et travailleurs du savoir ayant besoin de questions-réponses sur leurs propres documents — ainsi que l'administrateur, qui exploite le déploiement, gère les comptes et audite l'usage. Les parties prenantes secondaires incluent l'encadrant du projet, qui a joué le rôle de relecteur externe et a façonné la portée, et l'équipe de développement elle-même, qui s'est appuyée sur une construction reproductible et une boucle de déploiement rapide.

5.2. Besoins utilisateurs

Les besoins ont été recueillis à partir de l'usage que l'équipe faisait elle-même d'outils comparables, de courtes séances informelles de tests d'utilisabilité avec d'autres étudiants, et des revues de l'encadrant, puis traduits en éléments de backlog priorisés assortis de critères d'acceptation et validés lors des revues de sprint. Ils sont résumés ci-dessous sous forme de besoins fonctionnels et non fonctionnels.

5.2.1. Besoins fonctionnels

Les besoins fonctionnels sont résumés dans le Tableau 5.1.

Tableau 5.1 : Besoins fonctionnels.

ID	Besoin fonctionnel
BF-1	Un visiteur peut s'inscrire ; les utilisateurs authentifiés peuvent se connecter et se déconnecter.
BF-2	Un utilisateur peut envoyer un message texte et recevoir une réponse du modèle, les quinze derniers messages de la session étant conservés comme contexte.
BF-3	Un utilisateur peut choisir un persona qui ajuste le ton de l'assistant.
BF-4	Un utilisateur peut demander une image à partir d'un prompt textuel et la recevoir directement dans la conversation.
BF-5	Un utilisateur peut téléverser une image et demander à l'assistant de la retoucher (image-à-image).
BF-6	Un utilisateur peut joindre une image et obtenir l'extraction de son texte (OCR) et/ou sa description (légendage), le résultat servant de contexte.
BF-7	Un utilisateur peut téléverser un PDF et poser des questions auxquelles on répond à partir de son contenu (RAG), ou l'épingler comme contexte notebook durable.
BF-8	Un utilisateur peut enregistrer de l'audio et le faire transcrire en texte.
BF-9	Un utilisateur peut obtenir un rapport PDF généré à partir d'une réponse rédigée.
BF-10	Un utilisateur peut organiser ses sessions avec des étiquettes colorées, rechercher dans l'historique de conversation, et exporter une session au format PDF, Markdown ou JSON.
BF-11	Un utilisateur peut définir et rejouer des chaînes de workflows réutilisables à plusieurs étapes.
BF-12	Un utilisateur peut gérer son profil et son avatar (avec un recadreur dans le navigateur).
BF-13	Un administrateur peut se connecter à un tableau de bord distinct et consulter des statistiques en temps réel ainsi que des graphiques d'activité.
BF-14	Un administrateur peut rechercher, suspendre, réactiver, réinitialiser le mot de passe,

	supprimer et créer des comptes utilisateurs.
BF-15	Un administrateur peut parcourir l'historique de conversation de n'importe quel utilisateur regroupé par session.

5.2.2. Besoins non fonctionnels

Tableau 5.2 : Besoins non fonctionnels.

ID	Catégorie	Besoin
BNF-1	Performance	Les requêtes de chat et de CRUD principales doivent être traitées dans un budget de latence interactif sous charge normale ; les appels longs de modèle sont bornés par la limite d'itérations de l'agent n8n.
BNF-2	Sécurité	Les mots de passe doivent être stockés hachés avec bcrypt (12 tours de sel) ; les sessions doivent utiliser des JWT signés expirant après 7 jours ; le service d'administration doit rejeter tout jeton sans revendication is_admin.
BNF-3	Sécurité	Les réponses HTTP doivent porter les en-têtes de sécurité Helmet ; un rate limiting doit s'appliquer (auth 20/15 min, chat 30/min, admin 500/15 min) ; les téléversements sont plafonnés à 10 Mo et 5 pièces jointes par message.
BNF-4	Fiabilité	L'application doit s'auto-réparer au redémarrage via des migrations de base idempotentes et une réinitialisation de l'administrateur ; les conteneurs doivent utiliser des politiques de redémarrage pour qu'un redémarrage de l'hôte rétablisse le système.
BNF-5	Maintenabilité	La logique d'orchestration doit résider dans un workflow n8n visible, modifiable sans redéployer le code ; le code doit être modulaire par domaine fonctionnel.
BNF-6	Portabilité	L'ensemble du système doit démarrer avec une seule commande docker compose et doit être déployable sur un serveur virtuel vierge à l'aide d'un script.
BNF-7	Coût	Le système ne doit reposer que sur des APIs externes à offre gratuite et des composants auto-hébergeables.

5.3. Spécifications du système

Le back-end expose une API REST : les endpoints d'authentification et de profil sous /api/auth, les endpoints de chat et d'historique sous /api/chat, et les endpoints de documents, RAG, transcription, retouche d'image, YouTube et workflows sous /api/features ; le service d'administration expose un ensemble /api parallèle. Quelques endpoints sélectionnés sont listés dans le Tableau 5.3. Chaque message de chat est transmis au webhook n8n, qui renvoie un objet JSON contenant le texte de la réponse et une action optionnelle (generate_image ou generate_pdf). Les entités de données persistantes sont users, conversations (avec des champs optionnels image, PDF et pièce jointe), documents, document_chunks (avec embeddings stockés en texte), notebook_bindings et workflows ; elles sont détaillées dans la section 7.

Tableau 5.3 : Endpoints REST sélectionnés et leurs responsabilités.

Méthode et chemin	Service	Responsabilité
POST /api/auth/register · /login	Principal	Création de compte et connexion ; émet un JWT de 7 jours.
PUT /api/auth/profile · /password · /avatar	Principal	Mise à jour des champs du profil, changement du mot de passe, mise/retrait de l'avatar.
POST /api/chat/message	Principal	Chat principal : OCR/BLIP sur les images, enrichissement par l'historique et le persona, aller-retour n8n, génération d'image/PDF, persistance.
GET /api/chat/history · /sessions · /session/:id	Principal	Historique de conversation, sessions distinctes, et messages d'une session unique.
PUT /api/chat/sessions/:id/tag · GET /tags	Principal	Système de dossiers/étiquettes sur les sessions.
GET /api/chat/export/:id?format=pdf md json	Principal	Exporter une session dans le format choisi.
POST /api/features/documents · PUT /notebook	Principal	Téléverser un PDF (découpage + embeddings), le lier à une session en mode notebook ou RAG.
POST /api/features/transcribe · /image-edit · /youtube-transcript	Principal	Transcription audio (Whisper), retouche image-à-image, résumé YouTube.
GET/POST/PUT/DELETE /api/features/workflows	Principal	CRUD pour les chaînes de workflows définies par l'utilisateur.
GET /api/stats	Admin	Métriques du tableau de bord : totaux d'utilisateurs/messages/sessions/tokens/images/PDF/docs, activité quotidienne, utilisateurs les plus actifs.
GET/POST/PUT/DELETE /api/users[:id]	Admin	Lister, créer, modifier (statut/rôle), réinitialiser le mot de passe, supprimer des utilisateurs ; conversations par utilisateur.

6. Modélisation du système

6.1. Diagramme de contexte

La Figure 6.1 montre le modèle de contexte. La plateforme AstroBot (l'application web publique, le service d'administration et l'orchestrateur n8n) est le système étudié. Deux acteurs humains interagissent avec elle : l'utilisateur final, qui envoie des messages de chat, téléverse des fichiers et consomme les réponses et médias générés, et l'administrateur, qui gère les comptes et consulte les statistiques. La plateforme s'intègre à plusieurs systèmes externes : Mistral Cloud pour l'inférence LLM, le routeur HuggingFace Inference pour la génération d'images, le légendage et les embeddings, Groq Whisper pour la transcription, SerpAPI pour la recherche web en direct, et une base PostgreSQL partagée pour la persistance.

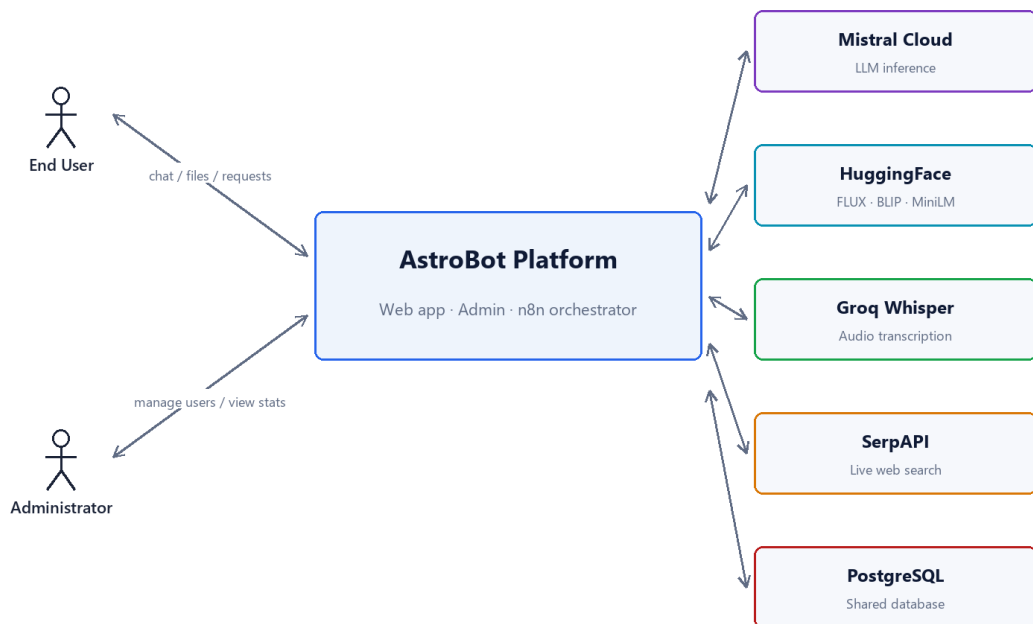


Figure 6.1 : Diagramme de contexte de la plateforme AstroBot.

6.2. Diagramme de cas d'utilisation

La Figure 6.2 résume les cas d'utilisation. L'utilisateur final peut s'inscrire et se connecter, envoyer des messages de chat, générer et retoucher des images, analyser des images via OCR et légendage, poser des questions sur un PDF via RAG, transcrire de l'audio, générer des rapports PDF, lancer des chaînes de workflows sauvegardées, exporter des

conversations et gérer son profil et son avatar. L'administrateur peut en outre consulter le tableau de bord de statistiques, gérer les comptes utilisateurs (recherche, suspension, réinitialisation, suppression, création) et auditer l'historique des conversations.

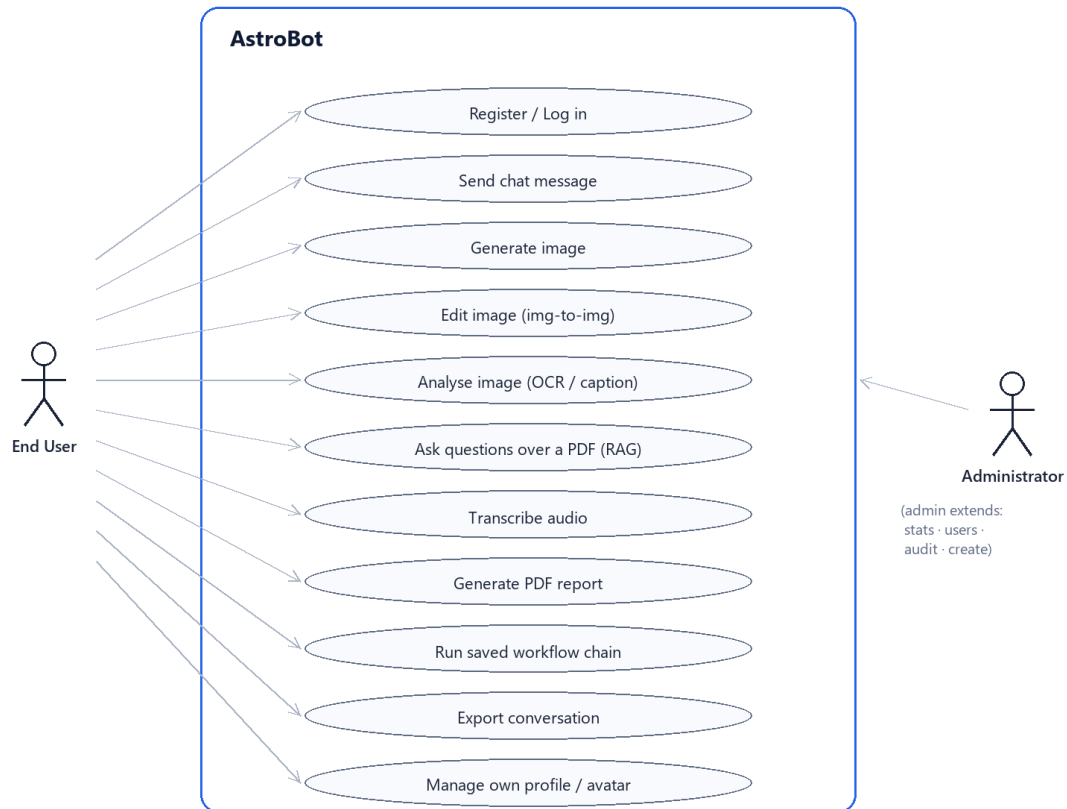


Figure 6.2 : Diagramme de cas d'utilisation (utilisateur final et administrateur).

6.3. Descriptions des cas d'utilisation

Deux cas d'utilisation représentatifs sont décrits ci-dessous.

Envoyer un message de chat — Acteur : utilisateur final authentifié. Préconditions : l'utilisateur dispose d'un JWT valide et se trouve sur la page de chat. Flux principal : l'utilisateur saisit un message, joint éventuellement des images et l'envoie ; le back-end exécute l'OCR et le légendage sur les images, charge les quinze derniers messages de la session, ajoute la directive du persona actif et tout contexte notebook/RAG, puis transmet la requête enrichie au webhook n8n ; l'AI Agent n8n appelle Mistral (avec recherche SerpAPI et un brouillon de raisonnement optionnels) et renvoie une réponse JSON ; le back-end déclenche la génération d'image ou de PDF si la réponse le demande, persiste la ligne de conversation et renvoie la réponse et les éventuels médias. Postconditions : l'historique de la conversation contient le nouvel échange. Flux alternatifs : si la sortie du

modèle ne peut pas être interprétée selon le contrat attendu, le back-end traite la sortie entière comme texte de réponse ; les erreurs de rate limit et de validation renvoient des réponses 4xx claires.

Poser des questions sur un PDF (RAG) — Acteur : utilisateur final authentifié.
Préconditions : l'utilisateur a téléversé un PDF et l'a lié à la session en mode RAG. Flux principal : à chaque question, le back-end calcule l'embedding de la question avec MiniLM, récupère les quatre fragments les plus similaires du document par similarité cosinus, les insère dans le prompt et continue comme dans le cas d'utilisation de chat. Postconditions : la réponse est ancrée dans les fragments récupérés. Flux alternatif : en mode notebook, les six mille premiers caractères du document servent de contexte à la place des fragments récupérés.

6.4. Diagrammes de séquence

La Figure 6.3 retrace l'aller-retour d'un message de chat. Le navigateur poste le message (avec d'éventuelles pièces jointes et le JWT) à l'application principale, qui charge l'historique de la session depuis PostgreSQL, poste un prompt enrichi au webhook n8n, où l'AI Agent appelle Mistral (et, le cas échéant, HuggingFace pour les images ou embeddings) ; n8n renvoie une réponse JSON structurée, l'application principale génère une image ou un PDF si demandé, insère la ligne de conversation et renvoie la réponse, les médias et les métadonnées au navigateur.

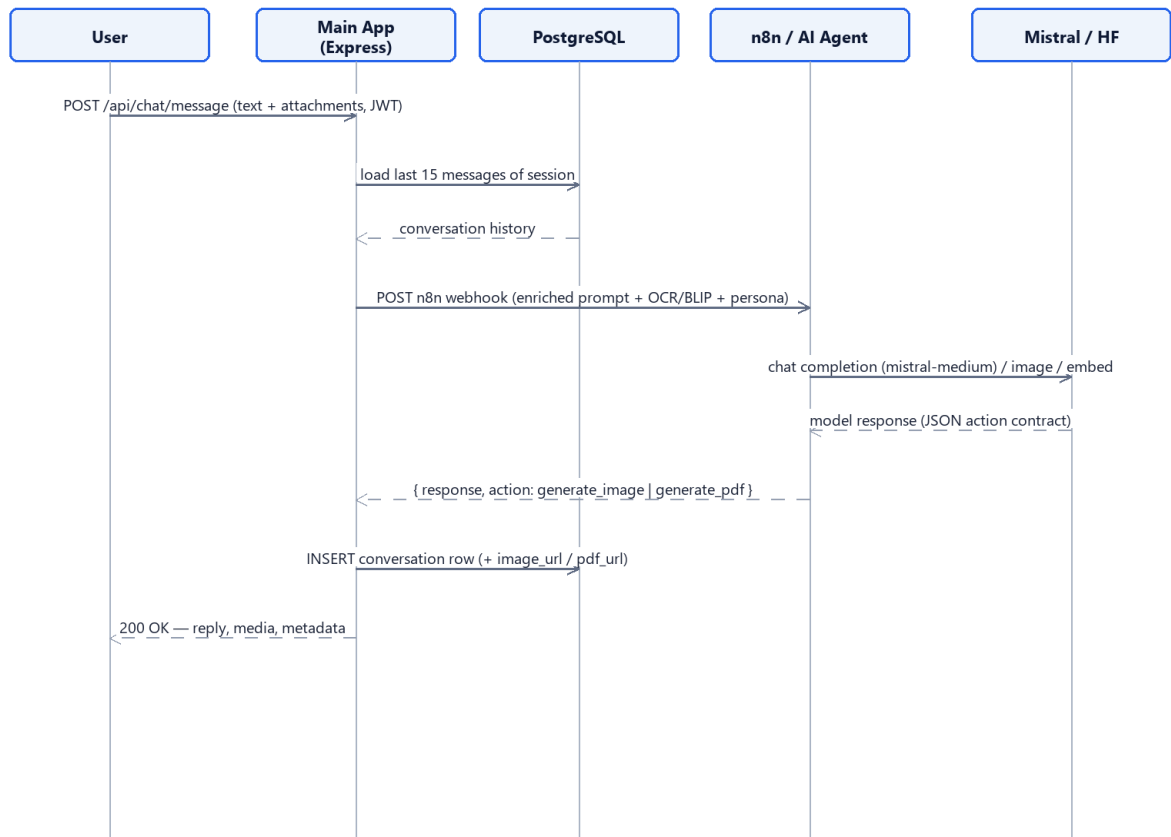


Figure 6.3 : Diagramme de séquence d'un aller-retour de message de chat.

7. Architecture et conception du système

7.1. Architecture du système

AstroBot suit une architecture client-serveur en couches, montrée dans la Figure 7.1, déployée sous forme de trois services Docker sur un même réseau privé. La couche client est le navigateur, qui sert l'interface publique de chat et celle du tableau de bord d'administration à partir de fichiers statiques. La couche application contient deux services Express indépendants : l'application principale sur le port 3000 (chat public, authentifié par JWT, avec Helmet et rate limiting) et le service d'administration sur le port 7040 (verrouillé par une revendication `is_admin`, complètement séparé du processus public). La couche d'orchestration est une instance n8n sur le port 5678 qui héberge le workflow de l'AI Agent et atteint Mistral, SerpAPI, un brouillon Think et un transcritteur Whisper auto-hébergé. La couche de données et de services externes se compose d'un conteneur PostgreSQL partagé (utilisé comme simple base relationnelle), du routeur HuggingFace Inference (FLUX, BLIP, MiniLM, avec un repli fal-ai) et des services externes Mistral, Groq et SerpAPI. Cette forme a été choisie pour que le conteneur PostgreSQL existant puisse être réutilisé sans exploiter une seconde base, que tout le trafic IA sorte par n8n afin de pouvoir ajuster les prompts et les outils sans redéploiement, et qu'une éventuelle exploitation publique ne puisse jamais atteindre le processus d'administration.

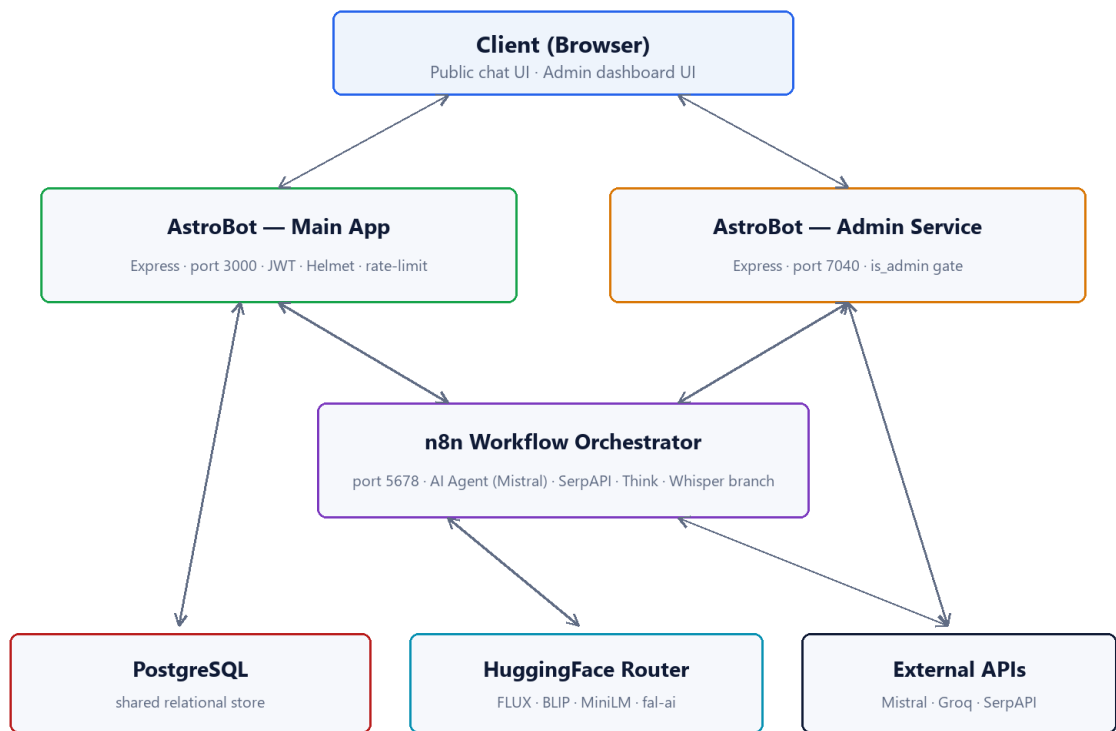


Figure 7.1 : Architecture du système (couches client / application / orchestration / données).

7.2. Modèles structurels

La conception statique est centrée sur le modèle de données persistant plutôt que sur une hiérarchie de classes profonde : le back-end est organisé par modules fonctionnels (auth, chat, features) avec des responsabilités de contrôleur, de service et d'accès aux données, et l'état durable réside dans un petit ensemble de tables relationnelles décrites ci-dessous et dans la Figure 7.2.

Tableau 7.1 : Principales tables de la base de données et leur rôle.

Table	Rôle
users	Comptes : nom, e-mail, mot de passe haché, is_admin, statut, avatar, last_login.
conversations	Une ligne par message/réponse, avec session_id, session_tag, et les champs optionnels image_url, pdf_url, pdf_filename et un blob JSON attachment.
documents	Fichiers téléversés pour notebook/RAG : nom, type MIME, taille, texte extrait text_content.
document_chunks	Fragments par document pour le RAG : chunk_index, chunk_text et un embedding stocké en texte.
notebook_bindings	Quel document est actif pour quelle session, et le mode (notebook rag).

workflows	Chaînes de prompts à plusieurs étapes définies par l'utilisateur, stockées comme tableau JSON steps.
-----------	--

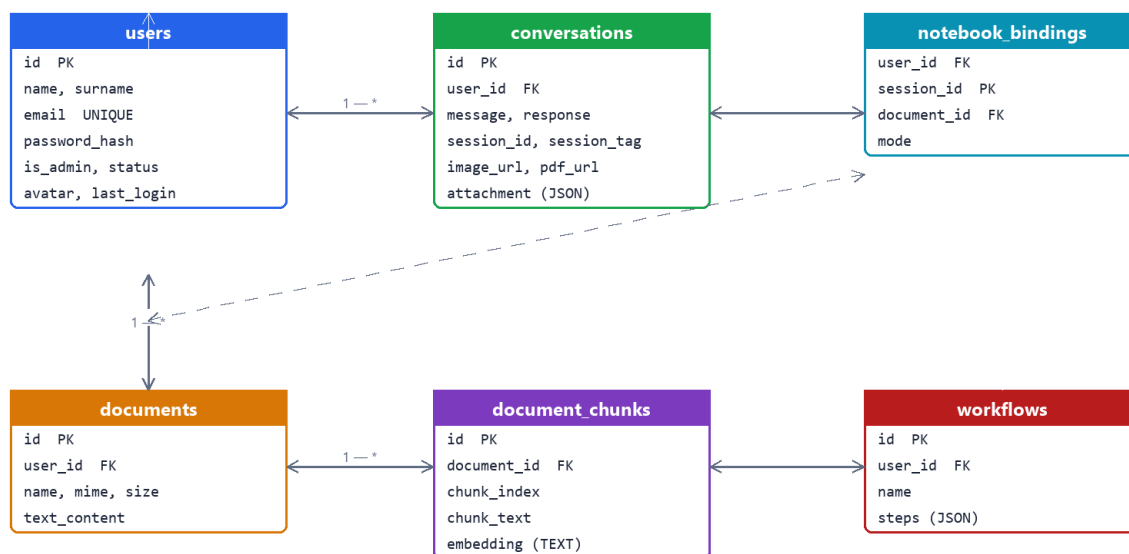


Figure 7.2 : Modèle entité-association du schéma PostgreSQL.

7.2.1. Diagramme de classes / entité-association

Comme le back-end est en JavaScript avec un accès SQL direct plutôt qu'un modèle de domaine orienté objet, la conception structurelle s'exprime au mieux par le diagramme entité-association de la Figure 7.2. Un utilisateur possède plusieurs conversations, plusieurs documents, plusieurs liaisons notebook et plusieurs workflows (un-à-plusieurs dans chaque cas) ; un document possède plusieurs fragments (un-à-plusieurs) ; et une liaison notebook relie la session d'un utilisateur à un document. Les clés étrangères cascaded à la suppression, de sorte que supprimer un utilisateur ou un document supprime ses lignes dépendantes.

7.2.2. Héritage (généralisation)

Le système utilise très peu d'héritage par conception. Conceptuellement, le type utilisateur est spécialisé en utilisateur ordinaire et administrateur, mais cette spécialisation est représentée par un booléen `is_admin` sur la table `users` unique plutôt que par une sous-

classe, ce qui maintient le schéma et la logique d'autorisation simples — le service d'administration exige simplement que `is_admin` soit vrai dans le JWT.

7.2.3. Agrégation / composition

L'agrégation et la composition apparaissent à deux endroits. Une session de conversation agrège ses lignes individuelles de messages (elles partagent un `session_id` mais chaque ligne reste indépendamment signifiante), et un document compose ses fragments : ceux-ci n'ont pas de sens hors de leur document parent et sont supprimés avec lui (composition, imposée par une clé étrangère `ON DELETE CASCADE`). Les liaisons notebook sont des objets d'association qui relient une session et un document avec un attribut de mode.

7.3. Modèles comportementaux

La conception dynamique est événementielle à la frontière d'orchestration et fondée sur des états pour un message individuel.

7.3.1. Modèle événementiel

Chaque message de chat est un événement posté au webhook `n8n`. À l'intérieur du workflow, un nœud Switch route l'événement par type (audio ou texte) ; la branche audio poste le fichier à un transcripteur Whisper et rejoint la branche texte ; un nœud de code enrichit la charge utile avec le nom de l'utilisateur et l'historique de conversation ; le nœud AI Agent la consomme et peut lui-même émettre des événements d'appel d'outil vers SerpAPI (plafonné à deux recherches) et vers un brouillon Think avant de produire sa réponse JSON, qu'un nœud final extrait et renvoie. Côté client, la réponse de l'assistant est révélée mot par mot, une animation pilotée par un timer plutôt que par un streaming côté serveur.

7.3.2. Diagramme d'états

La Figure 7.3 modélise le cycle de vie d'un message de conversation unique. Depuis l'état Idle, l'envoi d'un message fait passer la session en Processing ; si le modèle a besoin d'un outil ou d'une génération de média, la session entre dans Awaiting tool / media et revient à Processing dès l'arrivée du résultat ; lorsque la réponse est prête, la session passe à Replied ; l'envoi du message suivant la ramène à Processing, et la fermeture de la session est la transition terminale.

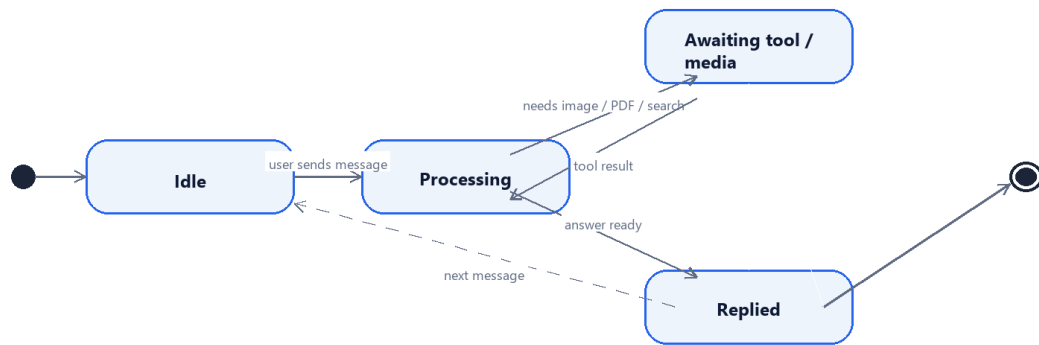


Figure 7.3 : Diagramme d'états d'un message de conversation.

8. Implémentation du système

8.1. Environnement de développement

Le développement a utilisé Git pour la gestion de versions avec un miroir public sur GitHub (<https://github.com/tkonate788/astrobot>), Visual Studio Code comme éditeur, Node.js 18 pour le back-end, et Docker Desktop / Docker Compose pour exécuter les services en local exactement comme en production. Le workflow n8n a été développé dans l'éditeur n8n et exporté en JSON conservé dans le dépôt. Le déploiement vers le serveur est effectué par un script Python (deploy.py) qui empaquette le projet, le téléverse par SFTP et exécute docker compose up -d --build à distance ; une suite Python de 66 cas d'intégration (test_all.py) est exécutée contre le déploiement en production.

8.2. Composants du système

Le MVP comprend quatre composants. L'application principale (front-end + back-end) sert l'interface publique de chat et l'API REST sur le port 3000. Le service d'administration sert l'interface du tableau de bord et une API parallèle sur le port 7040. L'instance n8n sur le port 5678 héberge le workflow d'orchestration. Le conteneur PostgreSQL partagé assure la persistance. En complément, le système dépend de plusieurs services tiers, résumés dans le Tableau 8.1. La Figure 8.1 montre le workflow n8n par lequel chaque message transite.

Tableau 8.1 : Services tiers utilisés par AstroBot.

Service	Usage	Comment il est appelé
Mistral Cloud (mistral-medium-latest)	LLM conversationnel et raisonnement de l'agent	À l'intérieur du nœud AI Agent de n8n
Routeur HuggingFace Inference	Génération d'images (FLUX.1-schnell), légendage d'images (BLIP), embeddings de phrases (MiniLM-L6-v2)	En HTTPS depuis le back-end / n8n
fal-ai	Génération d'images de repli et retouche image-à-image	En HTTPS depuis le back-end
Groq (Whisper-large-v3-turbo)	Transcription audio	En HTTPS depuis le back-end / la branche audio de n8n
SerpAPI	Recherche web en direct disponible pour l'agent (plafonnée à deux appels par message)	À l'intérieur de l'AI Agent n8n comme outil
PostgreSQL (conteneur partagé)	Stockage persistant pour toutes les entités	En TCP sur le réseau Docker privé

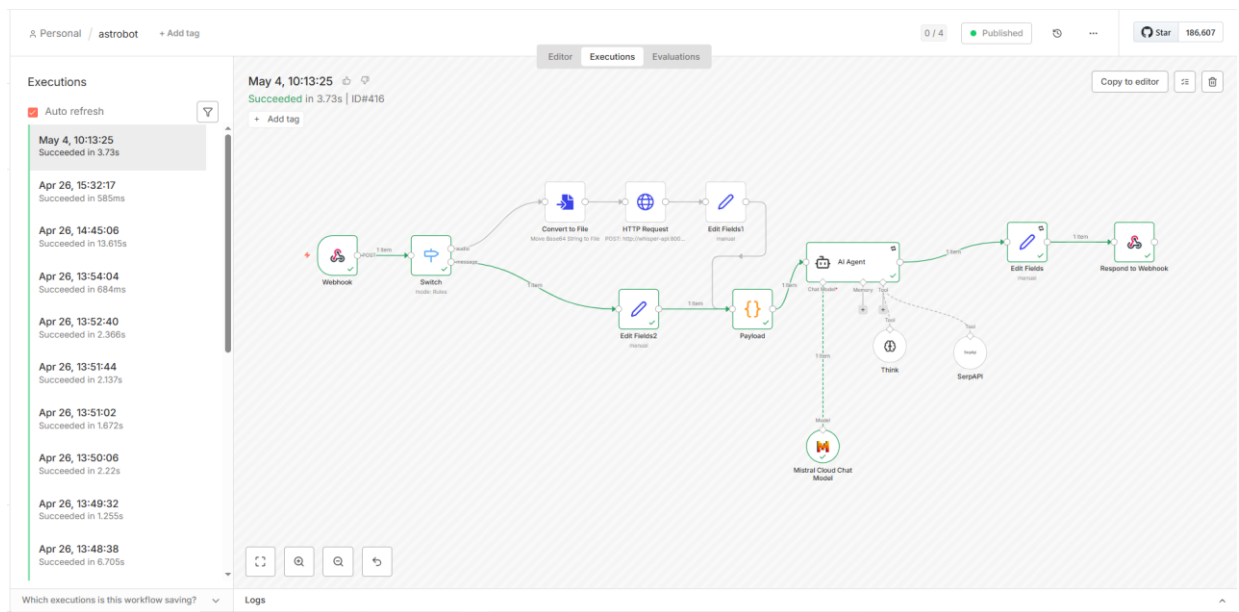


Figure 8.1 : Workflow n8n qui achemine chaque message via Mistral, avec recherche web et outils de raisonnement optionnels.

8.3. Structure du code

Le dépôt est organisé par composant et, au sein du back-end, par domaine fonctionnel :

```
astrobot/
├── frontend/                # static HTML / CSS / JS for the public app
│   ├── index.html chat.html login.html register.html
│   ├── style.css script.js
│   └── team/                # developer photos for the About section
├── backend/
│   ├── server.js            # Express entry point, middleware, routes
│   ├── db/index.js          # PostgreSQL pool + idempotent migrations + admin seed
│   ├── middleware/auth.js   # JWT verification
│   └── routes/
│       ├── auth.js          # register / login / profile / avatar
│       ├── chat.js          # messages, image gen, PDF, history, sessions, export
│       └── features.js       # documents, RAG, notebook, transcribe, YouTube, img-edit,
├── workflows
│   ├── admin/
│   │   ├── backend/server.js # separate Express service (port 7040)
│   │   └── backend/public/   # admin dashboard UI (vanilla JS + Chart.js)
│   └── docker/
│       ├── docker-compose.yml # the two AstroBot services on the n8n network
│       ├── Dockerfile         # main app
│       └── .env.example        # template for local secrets
├── deploy.py                 # SSH + SFTP + docker compose deployment
├── test_all.py               # 66-case integration test suite
└── README.md
```

Au sein du back-end, chaque module de routes sépare la gestion des requêtes des opérations effectuées, et le module de base de données détient l'ensemble du SQL et des migrations de schéma, afin que le schéma puisse évoluer en toute sécurité à chaque redéploiement. Des extraits de code représentatifs sont donnés en Annexe A.

9. Développement du MVP

9.1. Définition du MVP

Le MVP est défini comme le plus petit système déployable apportant une valeur réelle aux utilisateurs finaux et à un administrateur : un assistant de chat fonctionnel avec mémoire de conversation, les fonctionnalités multimodales et de récupération listées ci-dessous, des conversations persistantes et exportables, et un tableau de bord d'administration séparé — l'ensemble fonctionnant sur un serveur réel et couvert par une suite de tests automatisés. Tout ce qui dépasse ce périmètre (applications mobiles natives, facturation, modèle ajusté finement) est explicitement repoussé aux travaux futurs.

9.2. Fonctionnalités implémentées

Le MVP implémente les capacités suivantes :

- Chat conversationnel — Mistral medium-latest via l'AI Agent n8n, avec une fenêtre de contexte de quinze messages par session et sept personas sélectionnables.
- Rendu de contenu enrichi — Markdown, mathématiques KaTeX, coloration syntaxique du code par highlight.js et diagrammes Mermaid dans la bulle de l'assistant, avec une révélation mot par mot.
- Génération d'images — FLUX.1-schnell via le routeur HuggingFace avec un repli fallback, plus la retouche image-à-image (transfert de style) via FLUX dev.
- OCR et compréhension visuelle — OCR Tesseract.js (anglais et français) et légendage BLIP, les deux signaux étant ajoutés au prompt avant que le modèle ne réponde.
- Questions-réponses sur document — génération augmentée par la recherche sur des PDF téléversés (découpage, embeddings MiniLM, récupération top-4 par cosinus) et un mode notebook qui épingle le document comme contexte durable.
- Transcription audio — enregistrement dans le navigateur transcrit par Groq Whisper-large-v3.
- Génération de rapports PDF — réponses rédigées transformées en PDF téléchargeable (PDFKit), avec une heuristique qui force la génération lorsque le modèle reconnaît un rapport.
- Outils de productivité — étiquettes de session colorées et recherche, exportation de conversations (PDF / Markdown / JSON), résumeur YouTube, chaînes de workflows réutilisables, et éditeur de profil/avatar avec un recadreur dans le navigateur.

- Administration — un tableau de bord séparé avec des KPI et graphiques en temps réel, la gestion des utilisateurs (recherche, suspension, réactivation, réinitialisation du mot de passe, suppression, création) et l'audit des conversations par utilisateur, regroupées par session.

9.3. Interfaces du système

L'application publique s'ouvre sur une page d'accueil (Figure 9.1) avec une barre de chat centrale et une mascotte sympathique, une section fonctionnalités (Figure 9.2) et une page « À propos » présentant l'équipe (Figure 9.3). Après connexion, l'utilisateur atteint l'interface de chat (Figure 9.4) avec la barre latérale de sessions et les commandes de pièces jointes ; un sélecteur de personas (Figure 9.5) ajuste le ton de l'assistant, et un panneau de profil avec un recadreur d'image dans le navigateur (Figure 9.6) gère le compte. L'administrateur utilise un tableau de bord séparé avec des tuiles KPI et des graphiques d'activité (Figure 9.7), un tableau de gestion des utilisateurs (Figure 9.8) et un panneau de paramètres en libre-service (Figure 9.9).

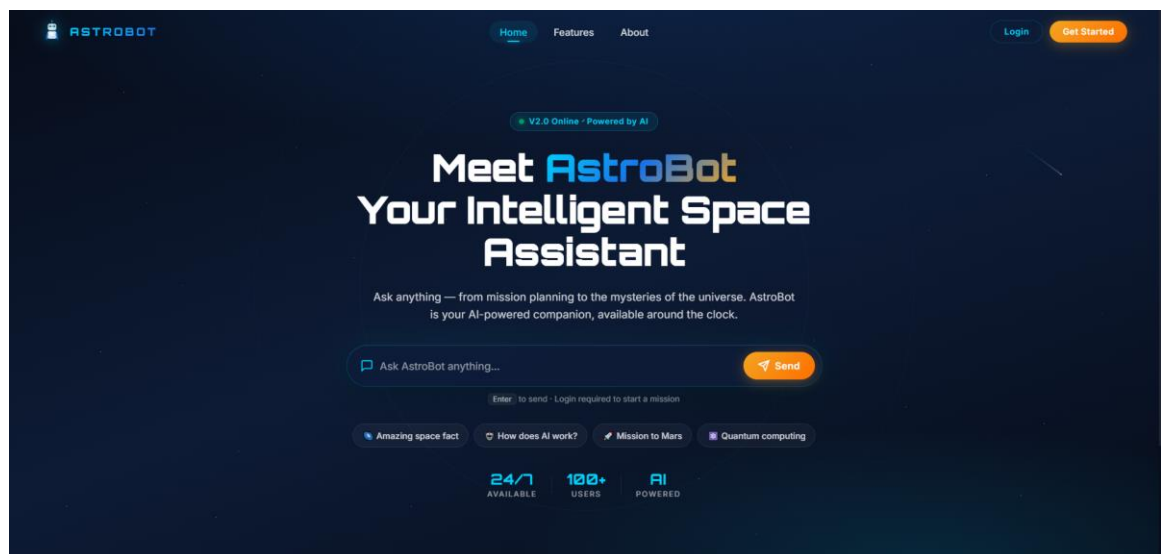


Figure 9.1 : Page d'accueil publique avec la barre de chat centrale.

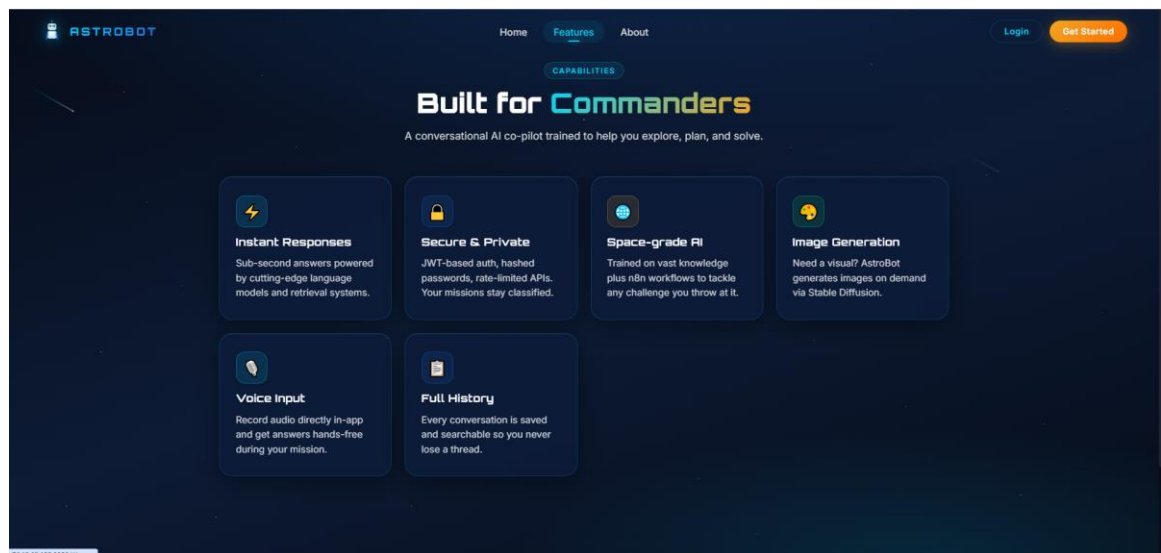


Figure 9.2 : Section fonctionnalités de l'application publique.

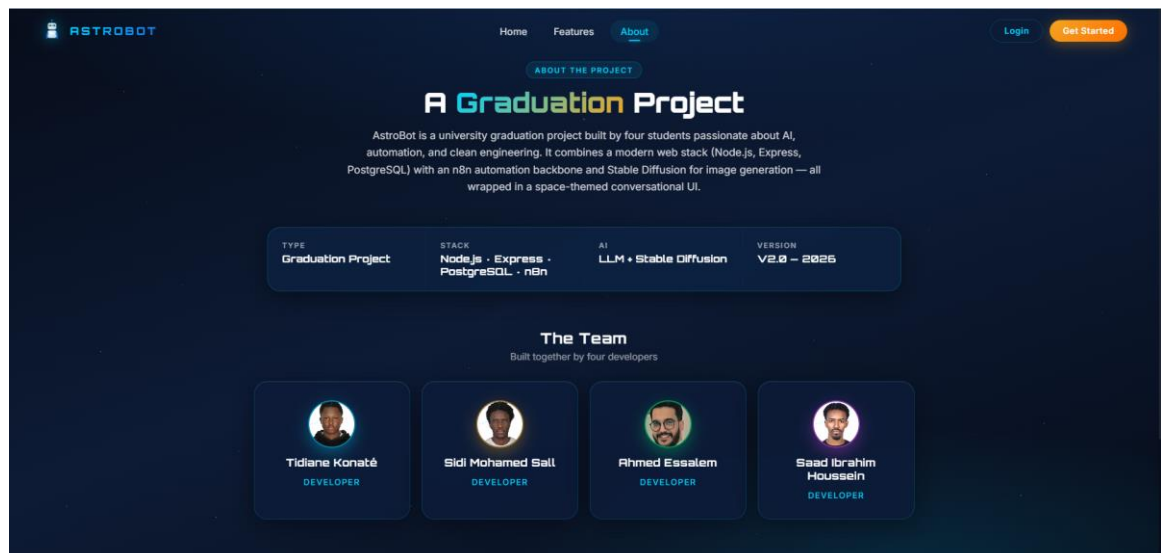


Figure 9.3 : Page « À propos » présentant l'équipe de quatre développeurs.

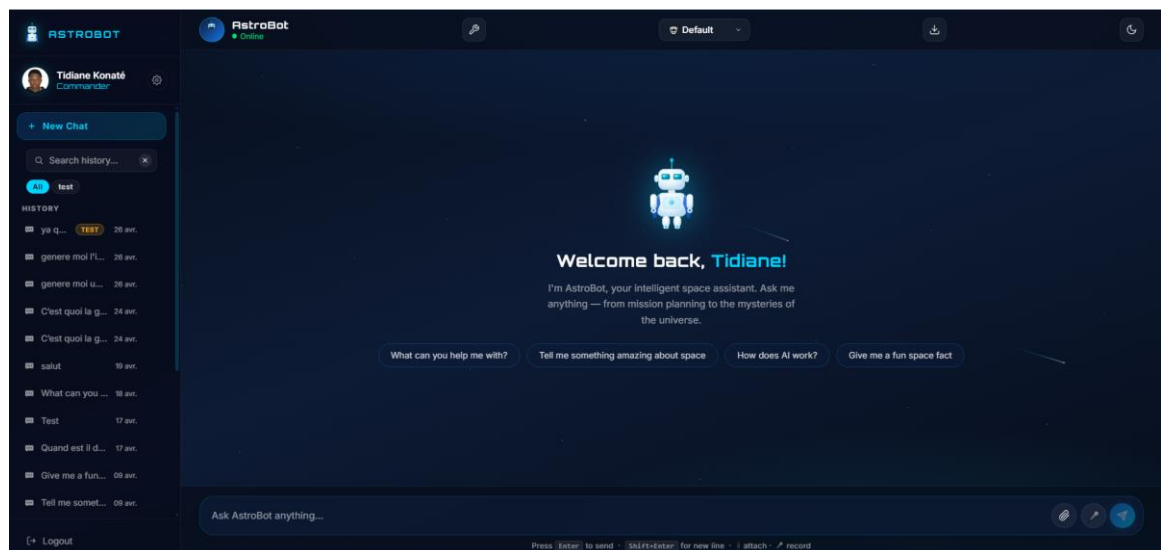


Figure 9.4 : Interface de chat authentifiée (écran d'accueil).

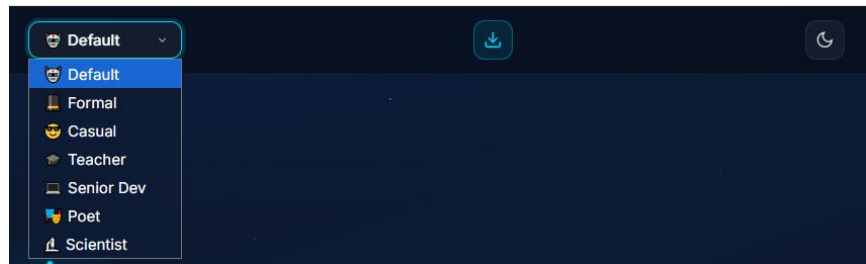


Figure 9.5 : Sélecteur de personas — sept personas d'assistant disponibles.

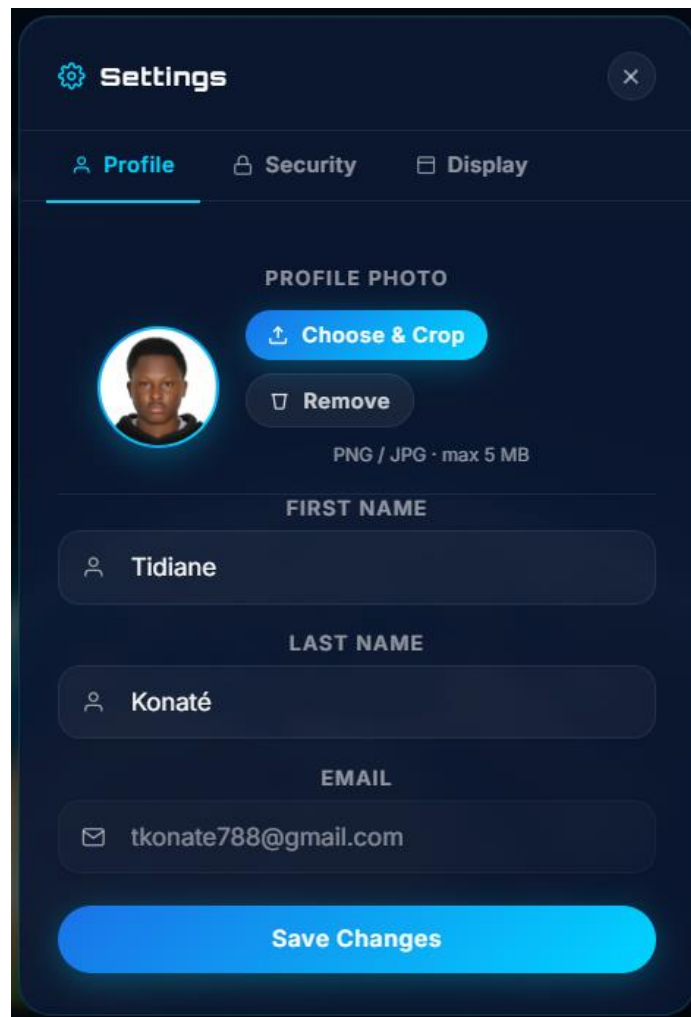


Figure 9.6 : Paramètres de profil avec un recadreur d'image dans le navigateur.

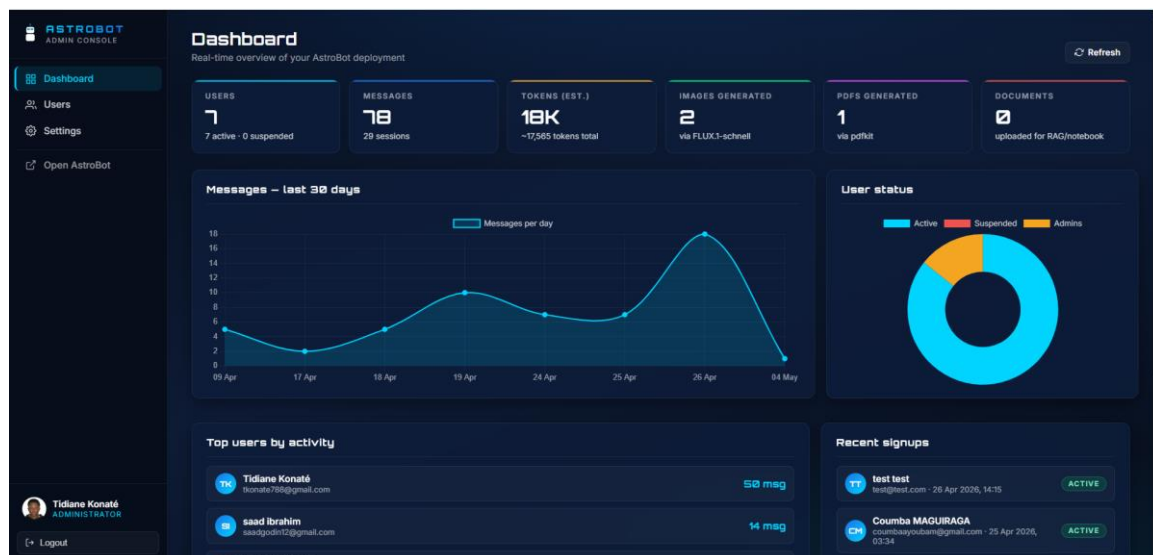


Figure 9.7 : Tableau de bord admin — KPI et graphiques d'activité.

The Users management page allows administrators to manage commanders, status, and permissions. It includes a search bar and a table of users.

USER	EMAIL	STATUS	ROLE	SESSIONS	MESSAGES	LAST LOGIN	JOINED	
test test ID #8	test@test.com	ACTIVE	USER	0	0	—	26 Apr 2026, 14:15	👁️ ⚙️ 🗑️
Coumba MAQUIRAGA ID #7	coumbaayoubam@gmail.com	ACTIVE	USER	1	2	—	25 Apr 2026, 03:34	👁️ ⚙️ 🗑️
Sidi Mohamed Sali ID #6	sidi Mohamedsali5@gmail.com	ACTIVE	USER	1	5	—	25 Apr 2026, 03:34	👁️ ⚙️ 🗑️
Mamadou Keita ID #5	mkeita5912@gmail.com	ACTIVE	USER	2	6	—	24 Apr 2026, 14:21	👁️ ⚙️ 🗑️
saad ibrahim ID #4	saadgodin12@gmail.com	ACTIVE	USER	4	14	—	18 Apr 2026, 20:04	👁️ ⚙️ 🗑️
Modi Niang ID #3	modiniang1975@gmail.com	ACTIVE	USER	1	1	—	2 Apr 2026, 00:42	👁️ ⚙️ 🗑️
Tidiane Konaté ID #2 - You	tkonate78@gmail.com	ACTIVE	ADMIN	20	50	4 May 2026, 10:20	12 Mar 2026, 15:54	👁️

Figure 9.8 : Tableau de gestion des utilisateurs (admin).

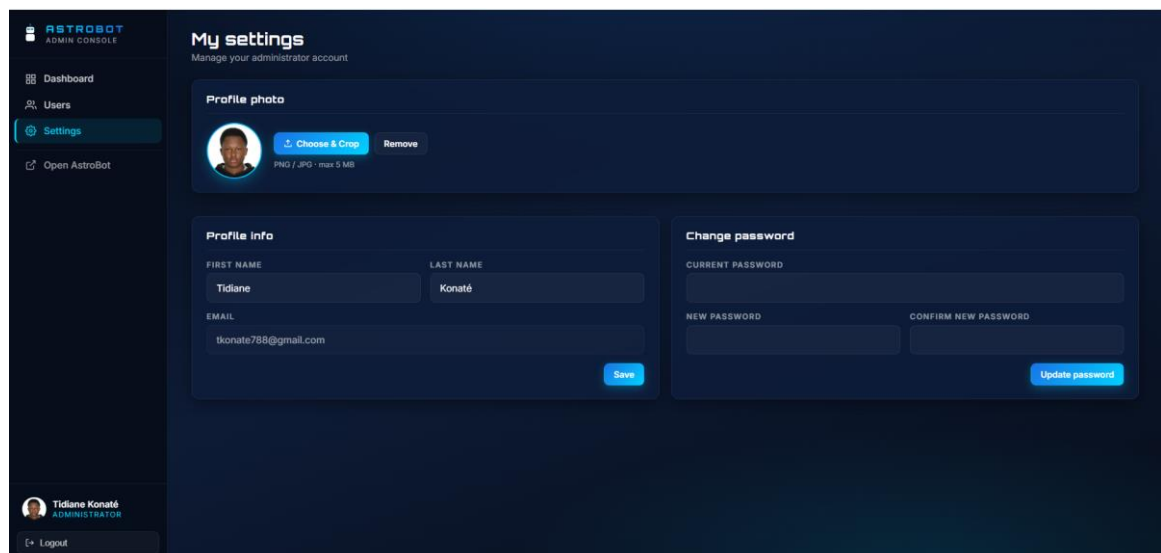


Figure 9.9 : Paramètres en libre-service de l'admin (profil et mot de passe).

9.4. Démonstration du système

Les fonctionnalités ont été démontrées contre le déploiement en production avec des scénarios représentatifs : une réponse riche en Markdown rendue dans la bulle de chat (Figure 9.10) ; la génération d'une image à partir d'un prompt (Figure 9.11) ; la transformation d'une réponse rédigée en rapport PDF téléchargeable (Figure 9.12) ; l'attachement d'une capture d'écran pour que l'assistant en lise le texte par OCR et l'explique (Figure 9.13) ; le téléversement d'un PDF et sa liaison comme contexte notebook/RAG (Figure 9.14) ; et la composition d'une chaîne de workflow réutilisable (Figure 9.15). La transcription audio a été démontrée en enregistrant un court extrait et en recevant la transcription en ligne.

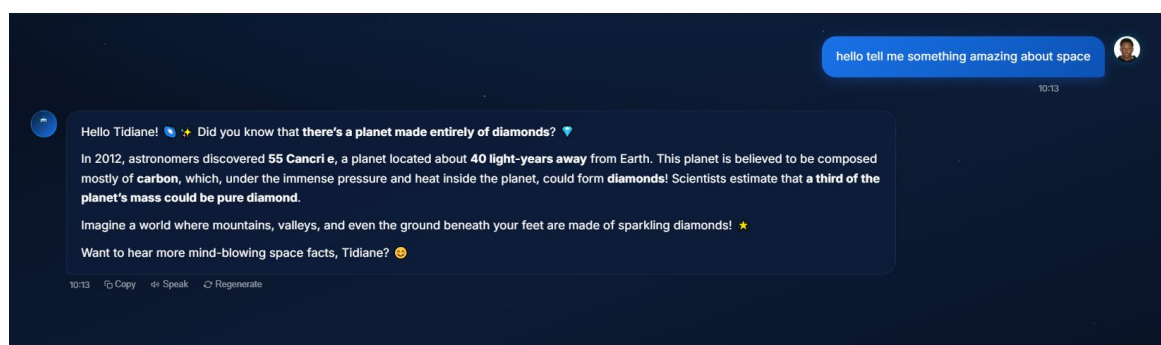


Figure 9.10 : Réponse riche en Markdown rendue dans une bulle de l'assistant.

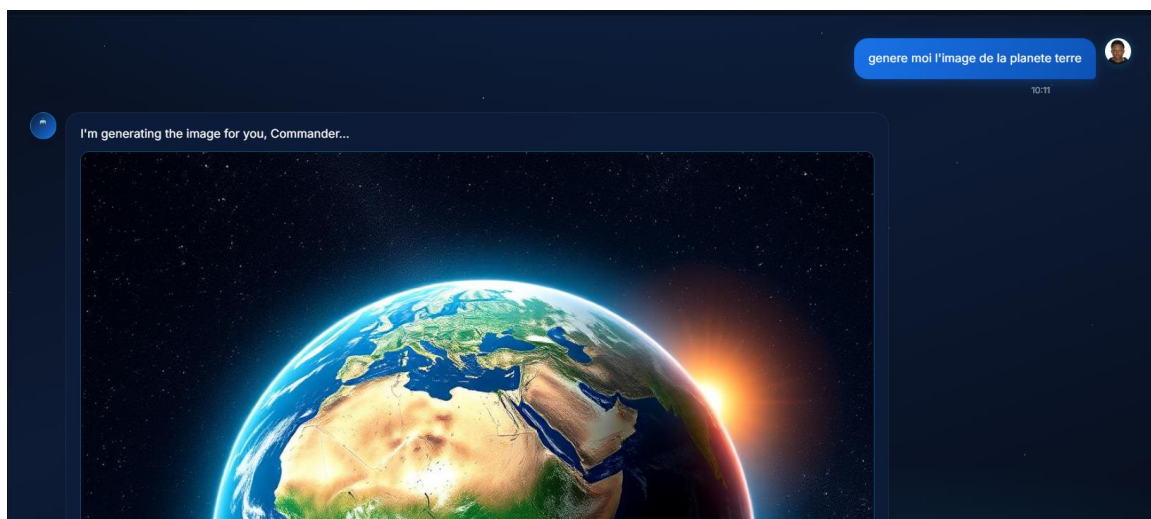


Figure 9.11 : Exemple de génération d'image par IA.

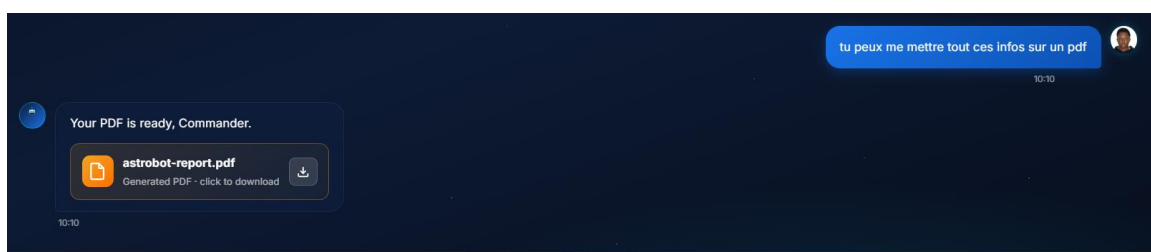


Figure 9.12 : Rapport PDF à la demande affiché comme carte de téléchargement.

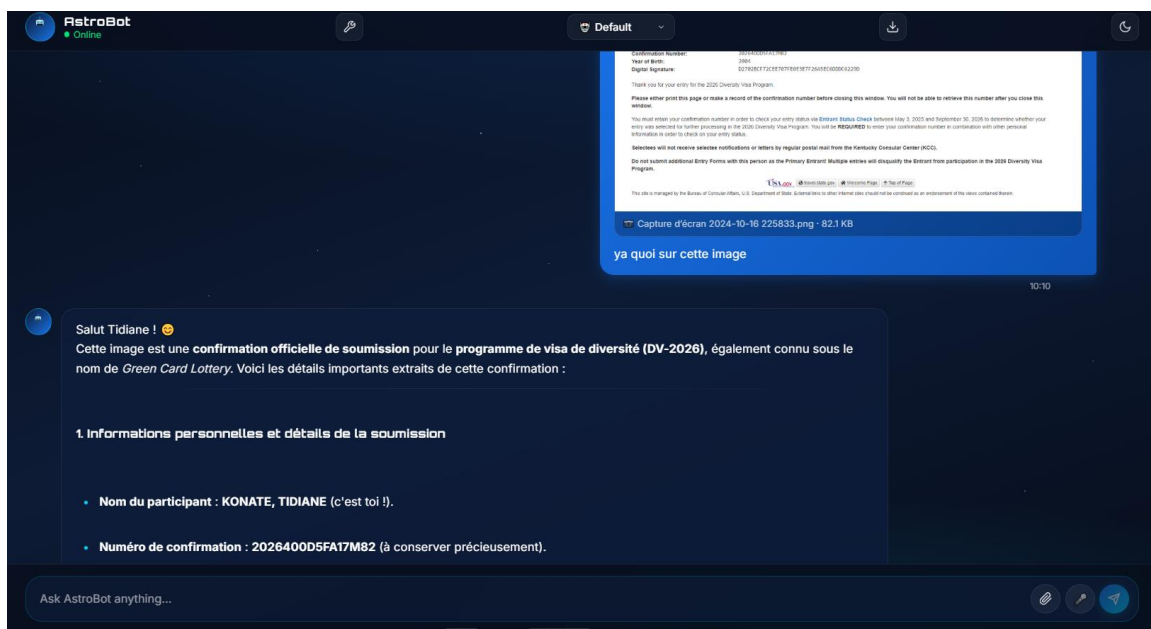


Figure 9.13 : Analyse OCR d'une image attachée.

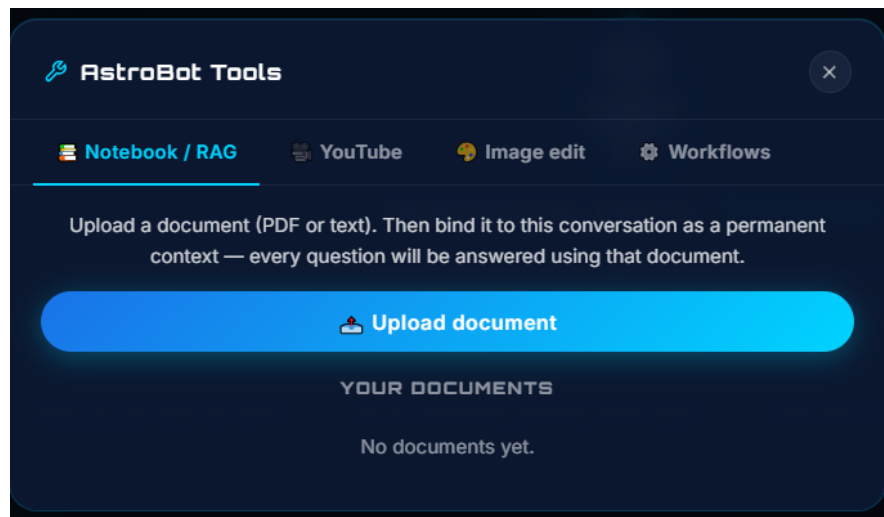


Figure 9.14 : Questions-réponses sur document — téléversement et liaison d'un PDF (notebook / RAG).

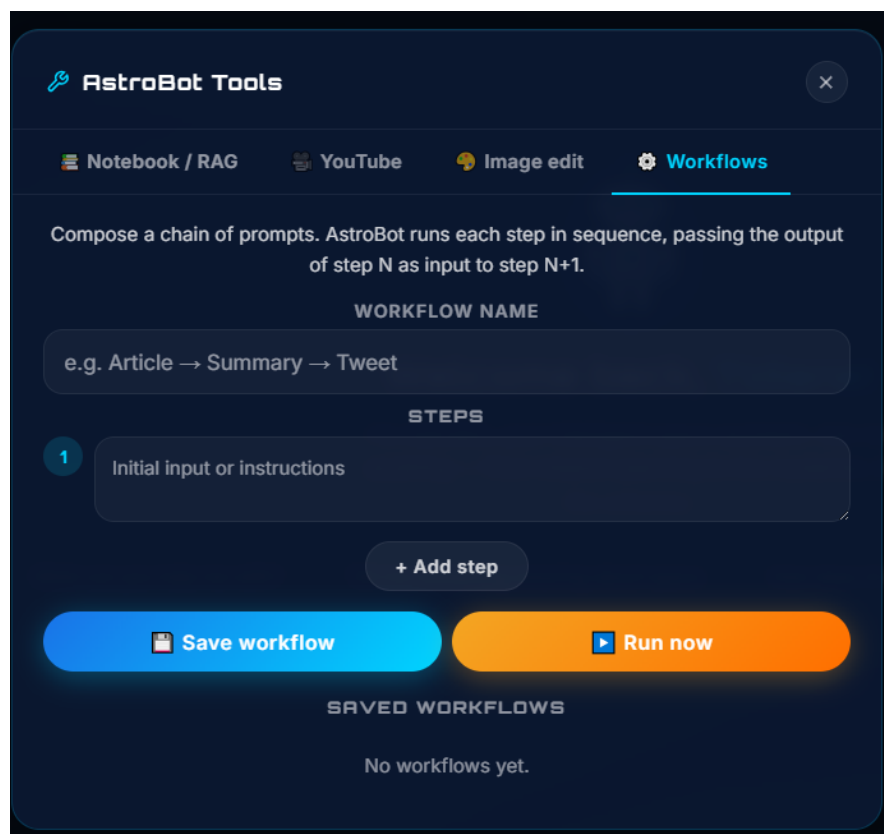


Figure 9.15 : Composition d'une chaîne de workflow réutilisable.

10. Tests et évaluation

10.1. Stratégie de tests

La stratégie de tests combine des tests d'intégration du système en cours d'exécution avec des contrôles d'acceptation basés sur des scénarios. L'instrument principal est `test_all.py`, une suite Python de 66 cas qui s'exécute contre le déploiement en production (application principale sur le port 3000 et service d'administration sur le port 7040), sollicitant de vraies requêtes HTTP, la base de données et l'aller-retour n8n plutôt que des mocks. En outre, chaque élément de backlog a été vérifié contre ses critères d'acceptation à la revue de sprint correspondante, et des séances informelles de tests d'utilisabilité ont fait émerger des améliorations de l'interface.

10.2. Scénarios de tests

Les scénarios représentatifs couverts par la suite incluent : la santé des deux services ; les fichiers statiques servis avec les bons en-têtes `no-cache` ; les flux d'authentification (identifiants corrects et incorrects, jeton manquant, vérification de rôle) ; l'ensemble de la surface CRUD d'administration (statistiques, liste, création, mise à jour, suspension/activation, réinitialisation du mot de passe, suppression) ; les opérations de profil et d'avatar de l'application principale et le changement de mot de passe ; l'aller-retour de chat via n8n et Mistral ; les étiquettes de session, l'historique, le détail de session et la suppression ; les trois formats d'export (PDF, Markdown, JSON) ; les endpoints document, bind/unbind RAG et CRUD workflows ; et la validation des entrées (requêtes mal formées renvoyant des réponses 4xx propres plutôt que 5xx).

10.3. Résultats des tests

Les 66 cas passent contre le déploiement en production. Le Tableau 10.1 résume les résultats par catégorie. Les quelques problèmes découverts durant le développement ont été corrigés avant ce rapport — notamment les fichiers statiques périmés sur iOS Safari (résolu avec des en-têtes de cache `no-store`) et le modèle reconnaissant occasionnellement un rapport PDF sans signaler l'action (traité en réanalysant la réponse et en forçant la génération). La limite principale de l'évaluation est l'échelle : les tests étaient fonctionnels et à petite échelle plutôt qu'un test de charge, ce qui est laissé aux travaux futurs.

Tableau 10.1 : Suite de tests d'intégration — résultats par catégorie.

Catégorie	Résultat
Santé des services (principal + admin)	réussi

Fichiers statiques et en-têtes no-cache	réussi
Authentification et contrôles de rôle	réussi
CRUD admin (stats, utilisateurs, cycle de vie)	réussi
Profil / avatar / mot de passe de l'app principale	réussi
Aller-retour de chat (n8n → Mistral)	réussi
Sessions, étiquettes, historique, suppression	réussi
Exports (PDF / Markdown / JSON)	réussi
Documents, bind/unbind RAG, workflows	réussi
Validation des entrées (4xx propre)	réussi
Total	66 / 66 réussis

11. Résultats et discussion

Le projet a produit un produit minimum viable déployé et fonctionnel qui atteint ses objectifs : une seule application web qui réunit l'IA conversationnelle avec la génération et la retouche d'images, l'OCR et le légendage, la transcription audio, les questions-réponses augmentées par la recherche, le contexte notebook, la génération de rapports PDF et les chaînes de workflows réutilisables, plus un tableau de bord d'administration séparé — le tout construit à partir d'APIs à offre gratuite et de composants auto-hébergeables, empaqueté avec Docker, déployé sur un serveur en production et couvert par une suite de 66 tests d'intégration qui passe intégralement. Le processus de type Scrum a bien fonctionné pour ce type de projet : planifier les fondations et le cœur conversationnel en premier a fait en sorte que chaque sprint ultérieur s'appuie sur une base stable, et les retours de l'encadrant ont alimenté directement le backlog. Acheminer tout le trafic IA via un workflow n8n visible s'est révélé précieux en pratique — les prompts, les limites d'outils et la branche audio ont été ajustés à plusieurs reprises sans redéployer le code applicatif. Les limites principales sont le compromis étendue contre profondeur (le système intègre des modèles accessibles plutôt que de concurrencer des modèles de pointe), la dépendance à la disponibilité et aux limites de débit des services externes gratuits, et la petite échelle de l'évaluation.

12. Conclusion et travaux futurs

12.1. Conclusion

AstroBot illustre un pipeline complet de génie logiciel — recueil des besoins, modélisation, architecture et conception, implémentation, tests et déploiement — appliqué à un problème pratique : la fragmentation des outils d'IA du quotidien. Le MVP résultant est en ligne, reproductible et utile, et il établit une architecture de base claire (une application publique, un service d'administration séparé, et une couche d'orchestration n8n sur une base de données partagée) sur laquelle d'autres capacités peuvent être ajoutées de manière incrémentale.

12.2. Travaux futurs

Les travaux futurs prévus comprennent une application web progressive (PWA) axée sur le mobile avec support hors-ligne et notifications push ; un mode vocal bidirectionnel (transcription continue en entrée, parole en sortie) ; une couverture OCR de plus de langues et un réglage des personas par langue ; le passage de tous les chemins audio à un service Whisper entièrement auto-hébergé ; une place de marché où les utilisateurs partagent et notent des chaînes de workflows ; des quotas et de la facturation optionnels par utilisateur pour un déploiement multi-tenants ; et une évaluation de performance et de fiabilité à plus grande échelle.

RÉFÉRENCES

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems, 2020.
- [2] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” Proc. EMNLP, 2020.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proc. EMNLP-IJCNLP, 2019.
- [4] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” Proc. ICLR, 2023.
- [5] K. Schwaber and J. Sutherland, “The Scrum Guide,” 2020. [En ligne]. Disponible : <https://scrumguides.org>
- [6] Mistral AI, “Mistral models documentation.” [En ligne]. Disponible : <https://docs.mistral.ai>
- [7] HuggingFace, “Inference API documentation.” [En ligne]. Disponible : <https://huggingface.co/docs/api-inference>
- [8] n8n GmbH, “n8n documentation.” [En ligne]. Disponible : <https://docs.n8n.io>
- [9] PostgreSQL Global Development Group, “PostgreSQL 16 documentation.” [En ligne]. Disponible : <https://www.postgresql.org/docs/>
- [10] Docker Inc., “Docker documentation.” [En ligne]. Disponible : <https://docs.docker.com>
- [11] OpenJS Foundation, “Express 4.x API reference.” [En ligne]. Disponible : <https://expressjs.com>
- [12] Dépôt source d'AstroBot. [En ligne]. Disponible : <https://github.com/tkonate788/astrobot>

CURRICULUM VITAE

Tidiane KONATÉ

INFORMATIONS PERSONNELLES

Nom complet : Tidiane KONATÉ

Nationalité : Malienne (Mali)

Lieu et date de naissance : 24 Mai 2004, à Bamako

E-mail : Tkonate788@gmail.com · 220201838@ostimteknik.edu.tr

FORMATION

Diplôme	Établissement	Année d'obtention
Lycée	Lycée Planete Enfants (LPE), Mali — Baccalauréat scientifique	2021
Formation linguistique en turc	Yozgat Bozok University (YOBÜ), Turquie	2022
Licence	OSTIM Technical University, Ankara — Génie informatique	2026 (prévue)

EXPÉRIENCE PROFESSIONNELLE

Institution	Rôle / activités	Année
INEDIA — Lille, France (Télétravail)	Stagiaire IA Générative appliquée / LLMOps — conception et déploiement de workflows d'automatisation alimentés par l'IA avec n8n, intégration d'APIs de LLM dans les processus métier, déploiement de services conteneurisés avec Docker, pipelines GenAI de bout en bout.	2025 – présent

DOMAINES DE COMPÉTENCE

- IA générative appliquée et APIs de LLM
- Automatisation et orchestration de workflows IA (n8n)
- DevOps / LLMOps — Docker, webhooks, intégration d'API
- Développement logiciel — C, C++, Python, C#
- Systèmes d'exploitation et réseaux

LANGUES ÉTRANGÈRES

Langue	Niveau de maîtrise
Français	Maternelle
Anglais	Avancé (C1)
Turc	Intermédiaire (B2)
Bambara	Courant

CERTIFICATS

AUTRES QUALIFICATIONS PERTINENTES

- Expérience pratique du déploiement de services d'IA conteneurisés et de workflows d'orchestration sur des serveurs distants.

PUBLICATIONS

PRÉSENTATIONS

Sidi Mohamed SALL

INFORMATIONS PERSONNELLES

Nom complet : Sidi Mohamed SALL

Nationalité : Malienne (Mali)

Lieu et date de naissance : 01 Septembre 2001, à Bamako

E-mail : Sidimohamedsall5@gmail.com · 210208992@ostimtechnik.edu.tr

FORMATION

Diplôme	Établissement	Année d'obtention
Lycée	Lycée Complexe Scolaire Kanouté Aïssé, Bamako, Mali	2020
Compétence en langue anglaise	OSTIM Technical University, Ankara	2021
Licence	OSTIM Technical University, Ankara — Génie logiciel	2026 (prévue)

EXPÉRIENCE PROFESSIONNELLE

—

DOMAINES DE COMPÉTENCE

- Vision produit, expérience utilisateur et conception d'interface
- Développement front-end en HTML, CSS et JavaScript
- Génie logiciel — analyse, conception et documentation
- Collaboration en équipe et travail agile

LANGUES ÉTRANGÈRES

Langue	Niveau de maîtrise
Français	Maternelle
Anglais	Avancé (C2)
Turc	Intermédiaire (B2)
Bambara	Courant

CERTIFICATS

—

AUTRES QUALIFICATIONS PERTINENTES

—

PUBLICATIONS

—

PRÉSENTATIONS

Ahmed ESSALEM

INFORMATIONS PERSONNELLES

Nom complet : Ahmed ESSALEM

Nationalité : Mauritanienne (Mauritanie)

Lieu et date de naissance : Mauritanie

E-mail : 220201992@ostimteknik.edu.tr

FORMATION

Diplôme	Établissement	Année d'obtention
Licence	OSTIM Technical University, Ankara — Génie informatique	2026 (prévue)

EXPÉRIENCE PROFESSIONNELLE

Institution	Rôle / activités	Année
OSTIM Technical University	C# : programmation orientée objet (POO) avancée, développement d'applications multi-modules, scripts, automatisation, intégration d'APIs et développement back-end.	2026
Datacamp	Python : programmation orientée objet (POO) avancée, développement d'applications multi-modules, scripts, automatisation, intégration d'APIs et développement back-end.	2024
Smart SA Company	Gestion de bases de données — bases SQL, modélisation de données, opérations CRUD et intégration en applications.	2025
Smart SA Company	Technologies web — développement front-end et back-end, applications web responsives et systèmes basés sur des APIs.	2024

DOMAINES DE COMPÉTENCE

- Génie informatique : architecture des systèmes, cycle de vie du développement logiciel (SDLC), résolution de problèmes, débogage et optimisation d'applications.
- Programmation Python : POO avancée, applications multi-modules, scripts, automatisation, intégration d'APIs et développement back-end.
- Programmation C# : POO avancée, applications multi-modules et développement back-end.
- Génie logiciel : conception d'applications, bonnes pratiques de code propre, gestion de versions avec Git/GitHub, tests et documentation.
- Gestion de bases de données : bases SQL, modélisation et intégration.
- Technologies web : applications web responsives et systèmes basés sur des APIs.

- Collaboration en équipe et résolution de problèmes : travail d'équipe agile, communication technique et raisonnement analytique.

LANGUES ÉTRANGÈRES

Langue	Niveau de maîtrise
Arabe	Courant
Français	Courant
Turc	Avancé (C1)
Anglais	Avancé (C1)

CERTIFICATS

—

AUTRES QUALIFICATIONS PERTINENTES

—

PUBLICATIONS

—

PRÉSENTATIONS

—

Saad Ibrahim HOUSSEIN

INFORMATIONS PERSONNELLES

Nom complet : Saad Ibrahim HOUSSEIN

Nationalité : Djiboutienne (Djibouti)

Lieu et date de naissance : Djibouti

E-mail : saadgodin2@gmail.com · 210201983@ostimtechnik.edu.tr

FORMATION

Diplôme	Établissement	Année d'obtention
Licence	OSTIM Technical University, Ankara — Génie logiciel	2026 (prévue)

EXPÉRIENCE PROFESSIONNELLE

Institution	Rôle / activités	Année
Datacomp	Développement Flutter — applications mobiles et web multi-plateformes avec Dart, UI/UX responsive, gestion d'état, intégration Firebase et connexion à des APIs REST.	2024
Datacomp	Gestion de bases de données — bases SQL, modélisation de données, opérations CRUD et intégration en applications.	2025
Datacomp	Technologies web — développement front-end et back-end, applications web responsives et systèmes basés sur des APIs.	2024

DOMAINES DE COMPÉTENCE

- Génie informatique : architecture des systèmes, cycle de vie du développement logiciel (SDLC), résolution de problèmes, débogage et optimisation d'applications.
- Programmation Python : POO avancée, applications multi-modules, scripts, automatisation, intégration d'APIs et développement back-end.
- Développement Flutter : applications mobiles et web multi-plateformes avec Dart, UI/UX responsive, gestion d'état, Firebase et APIs REST.
- Génie logiciel : conception d'applications, bonnes pratiques de code propre, gestion de versions avec Git/GitHub, tests et documentation.
- Gestion de bases de données : bases SQL, modélisation et intégration.
- Technologies web : applications web responsives et systèmes basés sur des APIs.
- Collaboration en équipe et résolution de problèmes : travail d'équipe agile, communication technique et raisonnement analytique.

LANGUES ÉTRANGÈRES

Langue	Niveau de maîtrise
Somali	Courant
Arabe	Courant
Français	Courant
Turc	Avancé (C1)
Anglais	Pré-avancé (B2)

CERTIFICATS

—

AUTRES QUALIFICATIONS PERTINENTES

—

PUBLICATIONS

—

PRÉSENTATIONS

—

13. Annexes

13.1. Annexe A : Code source

Des extraits représentatifs du code source suivent ; le code complet est disponible à l'adresse <https://github.com/tkonate788/astrobot>.

A.1 Middleware d'authentification JWT (backend/middleware/auth.js)

```
const jwt = require('jsonwebtoken');

const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];

  if (!authHeader) {
    return res.status(401).json({
      success: false,
      error: 'Access denied. No authorization header provided.',
    });
  }

  const parts = authHeader.split(' ');
  if (parts.length !== 2 || parts[0] !== 'Bearer') {
    return res.status(401).json({
      success: false,
      error: 'Invalid authorization format. Use: Bearer <token>',
    });
  }

  const token = parts[1];

  if (!token) {
    return res.status(401).json({
      success: false,
      error: 'Access denied. Token missing.',
    });
  }

  try {
    const secret = process.env.JWT_SECRET || 'change_me_in_env';
    const decoded = jwt.verify(token, secret);
    req.user = decoded;
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        error: 'Token expired. Please log in again.',
      });
    }
  }
  return res.status(403).json({
```

A.2 Pool de base de données et migrations idempotentes (backend/db/index.js)

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  host: process.env.DB_HOST || 'postgres_container',
  port: parseInt(process.env.DB_PORT) || 5432,
  database: process.env.DB_NAME || 'mydatabase',
```

```

user: process.env.DB_USER || 'tidiane',
password: process.env.DB_PASSWORD || 'changeme',
max: 20,
idleTimeoutMillis: 30000,
connectionTimeoutMillis: 5000,
});

pool.on('error', (err) => {
  console.error('[DB] Unexpected pool error:', err.message);
});

const initDB = async () => {
  const client = await pool.connect();
  try {
    console.log('[DB] Connected to PostgreSQL successfully.');
```

```

    await client.query(`
      CREATE TABLE IF NOT EXISTS users (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        surname VARCHAR(100) NOT NULL,
        email VARCHAR(255) UNIQUE NOT NULL,
        password_hash TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
    `);

    await client.query(`
      CREATE TABLE IF NOT EXISTS conversations (
        id SERIAL PRIMARY KEY,
        user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
        message TEXT NOT NULL,
        response TEXT,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
    `);

    // — Idempotent migrations: add session + media columns if missing —
    const CONV_COLUMN_ADDS = [
      ['session_id', 'VARCHAR(255)'],
      ['image_url', 'TEXT'],           // data URL of bot-generated image
      ['pdf_url', 'TEXT'],             // data URL of bot-generated PDF
      ['pdf_filename', 'VARCHAR(255)'],
      ['attachment', 'TEXT'],          // JSON: { name, mime, size, dataUrl }
    ];
    for (const [col, type] of CONV_COLUMN_ADDS) {
      await client.query(
        `ALTER TABLE conversations ADD COLUMN IF NOT EXISTS ${col} ${type};`
      );
    }

    // Add user avatar column (data URL of cropped profile photo)
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS avatar TEXT;`);
  }
}

```

A.3 Point d'entrée Express et middleware de sécurité (backend/server.js)

```

require('dotenv').config();

const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const path = require('path');
const { initDB } = require('./db/index');
```

```

const authRoutes = require('./routes/auth');
const chatRoutes = require('./routes/chat');
const featuresModule = require('./routes/features');
const featuresRoutes = featuresModule.router;

const app = express();
const PORT = process.env.PORT || 3000;

// — Security Middleware —————
app.use(
  helmet({
    contentSecurityPolicy: false,
    crossOriginEmbedderPolicy: false,
  })
);

// — CORS —————
app.use(
  cors({
    origin: true,
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization'],
  })
);

// — Rate Limiting —————
const globalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 200,
  standardHeaders: true,
  legacyHeaders: false,
  message: {
    success: false,
    error: 'Too many requests. Please try again later.',
  },
});

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 20,
  standardHeaders: true,
  legacyHeaders: false,
  message: {
    success: false,
    error: 'Too many authentication attempts. Please try again later.',
  },
});

```

13.2. Annexe B : Backlog des sprints

La Figure 13.1 montre la chronologie du projet ; les objectifs des sprints et les incréments livrés sont listés dans le Tableau 4.2 de la section 4. Chaque sprint maintenait un backlog de récits utilisateurs avec une priorité et des critères d'acceptation ; les éléments non terminés dans un sprint étaient repriorisés pour le suivant, et les retours de revue de l'encadrant étaient ajoutés au backlog comme nouveaux éléments.

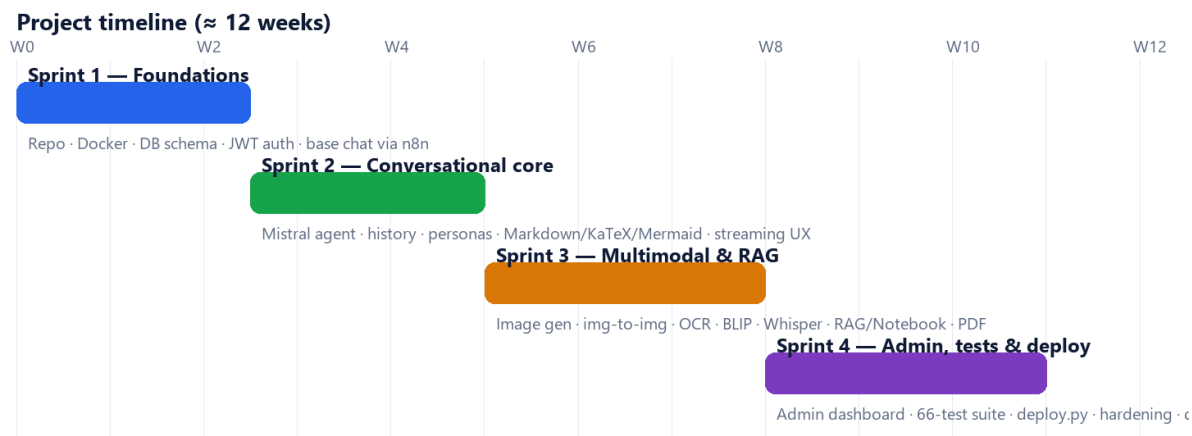


Figure 13.1 : Chronologie des sprints du projet.

Sprint	Récits utilisateurs sélectionnés (résumé)	Statut
1	Mettre en place le dépôt et Docker Compose ; créer le schéma de la BD ; implémenter l'inscription/connexion avec JWT ; chat minimal via n8n	Terminé
2	AI Agent Mistral ; historique de conversation (15 messages) ; 7 personas ; Markdown/KaTeX/highlight.js/Mermaid ; révélation de style streaming	Terminé
3	Génération d'images et img-à-img ; OCR + BLIP ; transcription Whisper ; RAG (découpage/embedding/récupération) et mode notebook ; génération de rapports PDF ; chaînes de workflows ; étiquettes et exports	Terminé
4	Tableau de bord admin (stats, utilisateurs, audit) ; suite de 66 tests d'intégration ; deploy.py ; en-têtes de sécurité et rate limits ; documentation	Terminé

13.3. Annexe C : Guide utilisateur

Instructions de démarrage rapide pour l'application déployée :

- Application publique — ouvrez <http://76.13.62.195:3000>, inscrivez-vous (ou connectez-vous) et démarrez une conversation depuis la barre de chat centrale. Utilisez le sélecteur de personas pour changer le ton de l'assistant, le bouton de pièce jointe pour ajouter des images (analysées par OCR/BLIP) ou un PDF (puis choisissez le mode notebook ou RAG), et le bouton micro pour dicter un message. Demandez une image (« génère une image de ... ») ou un rapport (« fais-moi un rapport PDF de ... ») et le résultat apparaît en ligne. Étiquetez, recherchez et exportez les sessions depuis la barre latérale.

- Administrateur — ouvrez `http://76.13.62.195:7040` et connectez-vous avec un compte administrateur. Le tableau de bord affiche les KPI et graphiques d'activité ; la page Utilisateurs permet de rechercher, suspendre, réactiver, réinitialiser le mot de passe, supprimer ou créer des comptes ; ouvrir un utilisateur affiche son historique de conversation regroupé par session.
- Déploiement — depuis le dépôt, copiez `docker/.env.example` vers `docker/.env` et renseignez les secrets, puis exécutez `docker compose up -d --build` dans le dossier `docker/` ; ou exécutez `python deploy.py` pour déployer sur le serveur configuré. Exécutez `python test_all.py` pour lancer la suite d'intégration contre le déploiement.
- Dépannage — si l'application publique est injoignable, vérifiez que le conteneur PostgreSQL partagé est en cours d'exécution (`docker ps`) ; les services AstroBot redémarrent automatiquement au redémarrage de l'hôte, mais la base de données doit elle aussi être en marche pour qu'ils puissent commencer à servir.